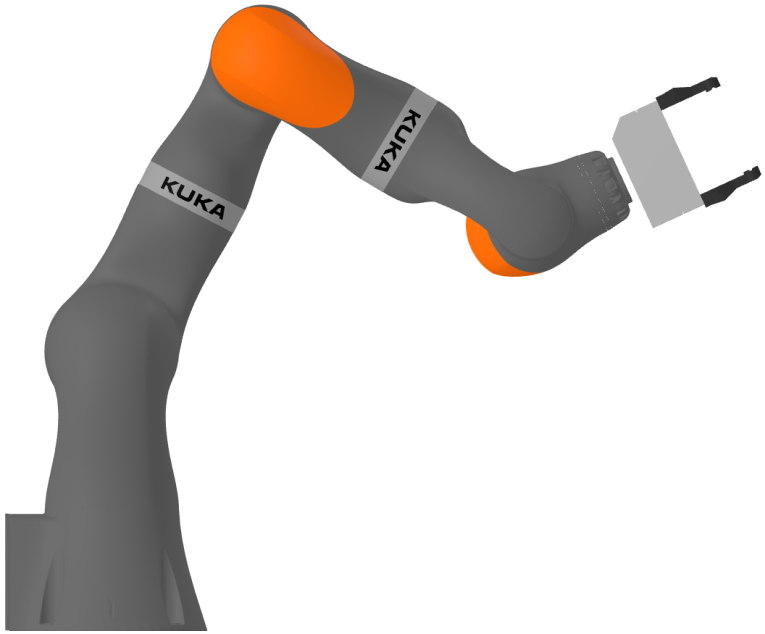


Task and Motion Planning

MIT 6.4210/2:

Robotic Manipulation

Fall 2025, Lecture 21

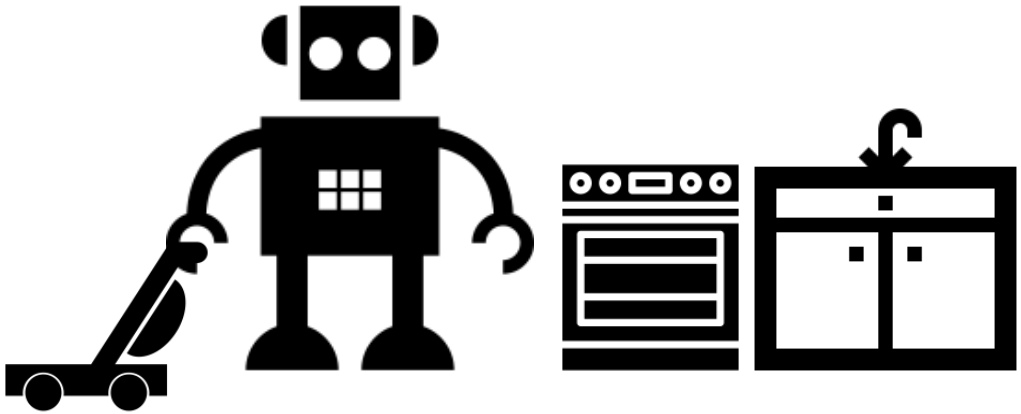
Massachusetts Institute of Technology

Task and Motion Planning (and learning) for general robots

Tomas Lozano-Perez
Leslie Pack Kaelbling

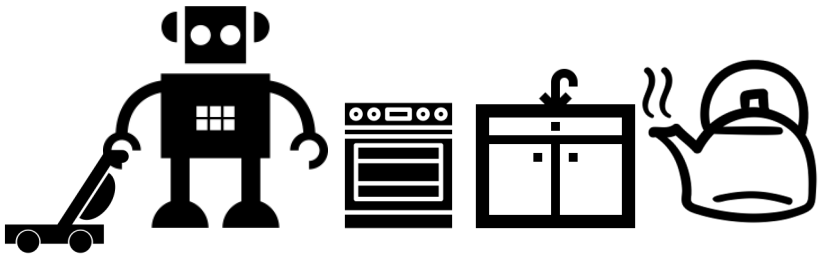
with LIS group

Research goal:
understand the computational mechanisms
necessary to make
a general-purpose intelligent robot



What makes this hard?

- Enormous variability in task demands and environments (countries, houses, families, jobs)
- Enormous state space (miles, millions of objects)
- Long horizon (years)





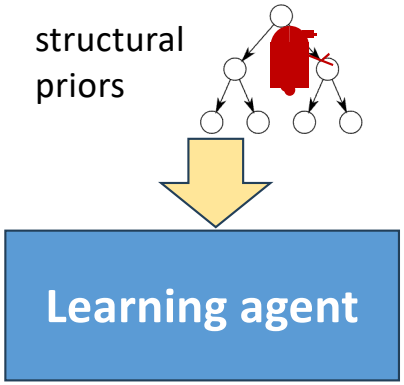
An agent is general to the extent that

- it can operate in **highly variable** circumstances
- it can achieve a **wide variety** of objectives
- it can **recover robustly** from failure or changing circumstance
- it is "**educable**" : can be taught new concepts (words, actions, etc.) with small amounts of computation and data

Generality via a #$(*&-ton of data

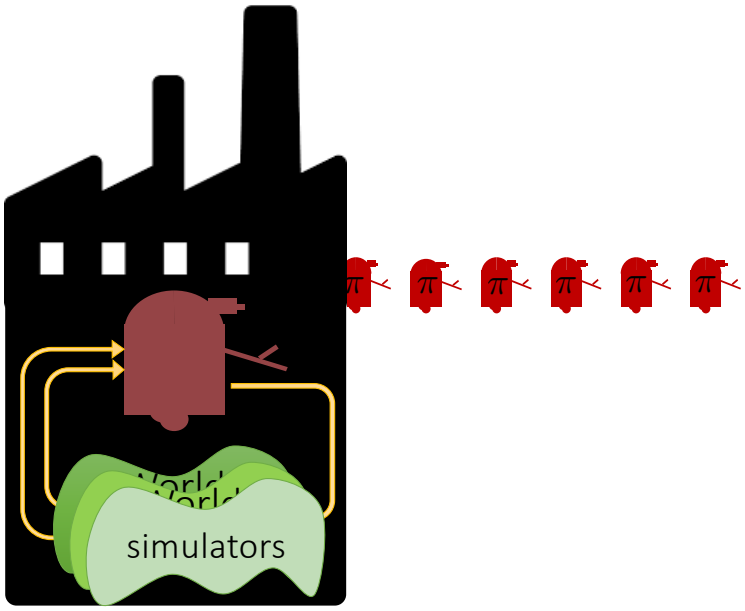


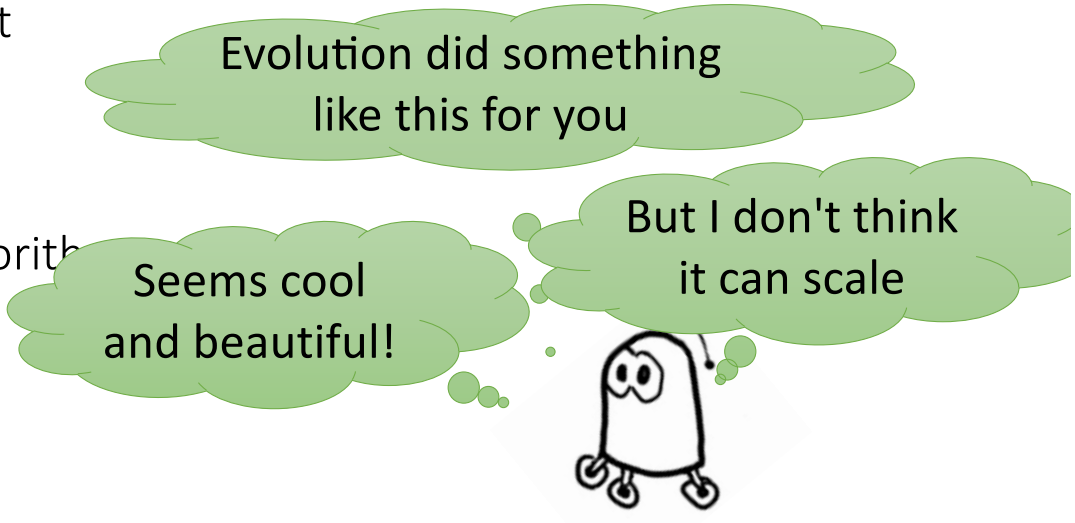
Generality via generalization



Main RL current paradigm

- Build a distribution of world simulators that mirrors the distribution of worlds (objects, houses, landscapes, humans) our robots could ever possibly encounter
- Design a completely general-purpose substrate for (meta) behavior learning
- Implement a **very very large** instance of that learning method
- Train it until it achieves general-purpose intelligence, inventing the best possible algorith for the problem distribution





Pragmatic alternative

Take all possible advantage of:

- the structure of the physical (and social) world
- algorithmic insights from computer science and cognitive science
  - abstraction, composition
- our accumulated cultural knowledge

We believe

1. There is an external physical reality with
   - sparse and local dynamics
   - sparse and local observations

2. Independence structure in computational problems often enables
   - efficient solution
   - compositional representation
   - sample-efficient learning
   - non-destructive incremental learning

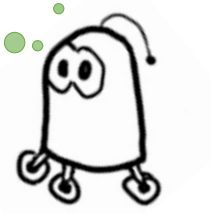
3. Causal models of transitions and observations have more independence structure than policies and are therefore better targets for learning

So, let's engineer **general-purpose** rational robots

1. There is an external physical reality with
   - sparse and local dynamics
2.
   - Start with built-in fundamental learning and inference algorithms
   - Build and pre-train general, compositional world models at design time
   - Continually and compositionally improve and augment these models at runtime
   - Perform runtime rational inference to select actions
3. ture

May be more like the way humans work!

May be easier for humans to understand



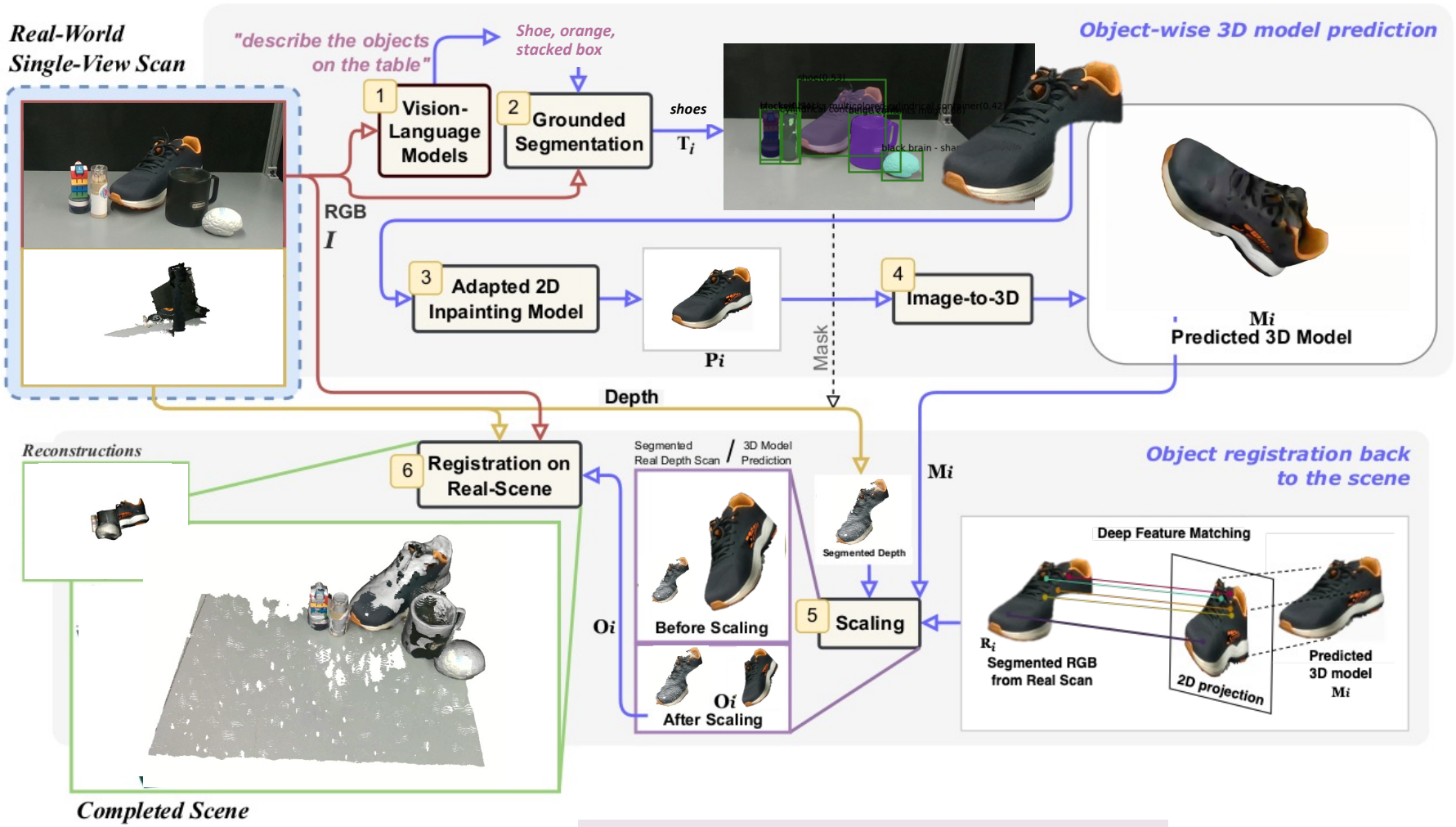
5 key ideas

1. There is an external physical reality that is worth believing in
2. World-model + planning algorithm = policy
3. Pick representations for learning based on sample efficiency / generalization
4. Learning can make planning better
5. Planning can make learning better

5 key ideas

1. There is an external physical reality that is worth believing in
2. World-model + planning algorithm = policy
3. Pick representations for learning based on sample efficiency / generalization
4. Learning can make planning better
5. Planning can make learning better

Object-based scene understanding: no prior object models



Aditya Agarwal, et al, SceneComplete, RAL

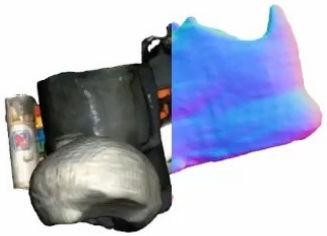
SceneComplete: Open-World 3D Scene Completion in Complex Real-World Environments for Robot Manipulation

Aditya Agarwal[1], Gaurav Singh[2], Bipasha Sen[1],
Tomás Lozano-Pérez[1], Leslie Pack Kaelbling[1]

[1]MIT CSAIL, [2]IIIT-Hyderabad







5 key ideas

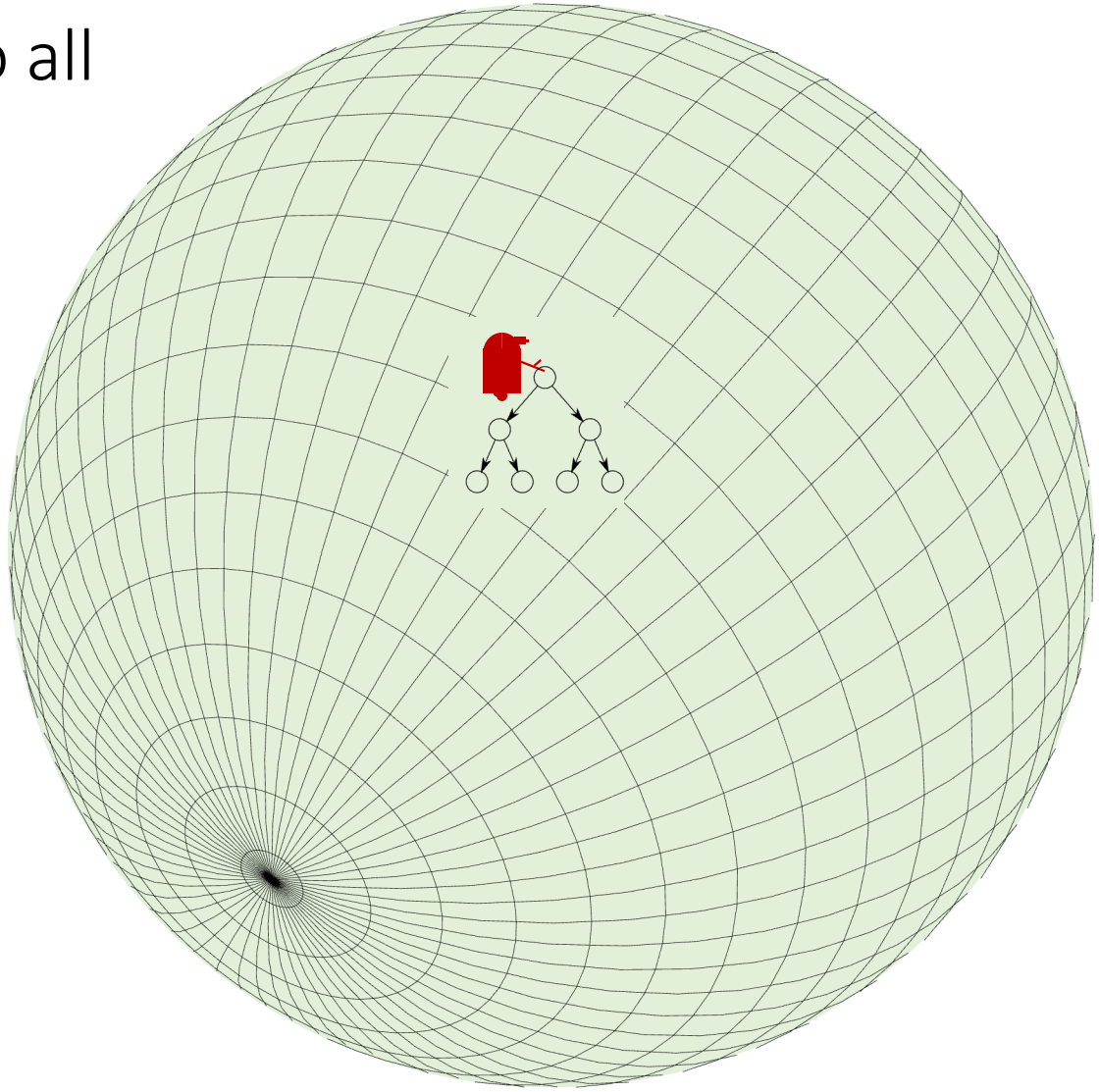
1. There is an external physical reality that is worth believing in
2. World-model + planning algorithm = policy
3. Pick representations for learning based on sample efficiency / generalization
4. Learning can make planning better
5. Planning can make learning better

Policies contain answers to all possible problems

Hard to compute!

Huge to store!

The set of situations any individual actually encounters in its lifetime is orders of magnitude smaller than set of possible situations



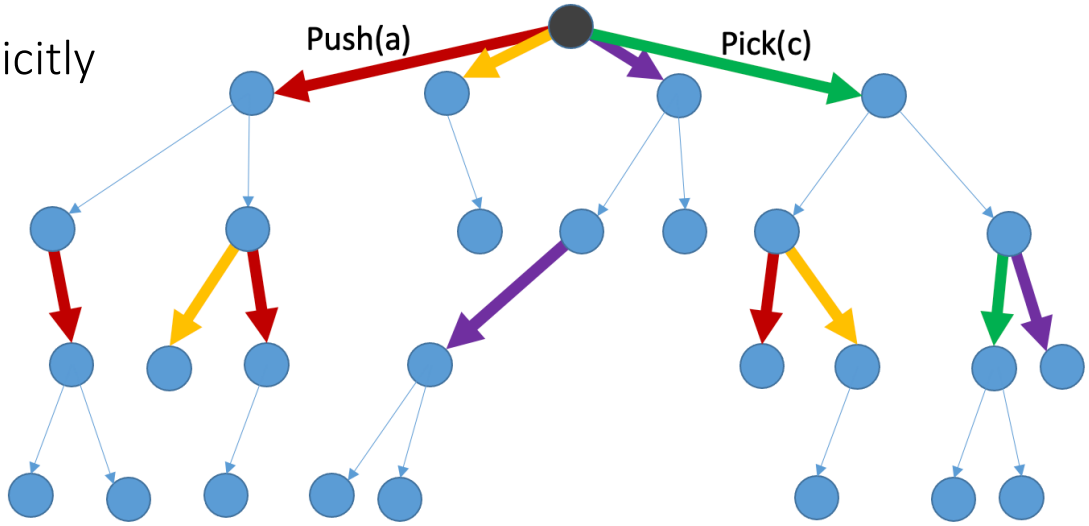
Planning leverages runtime inference

Performing computation at runtime may make action-selection slower

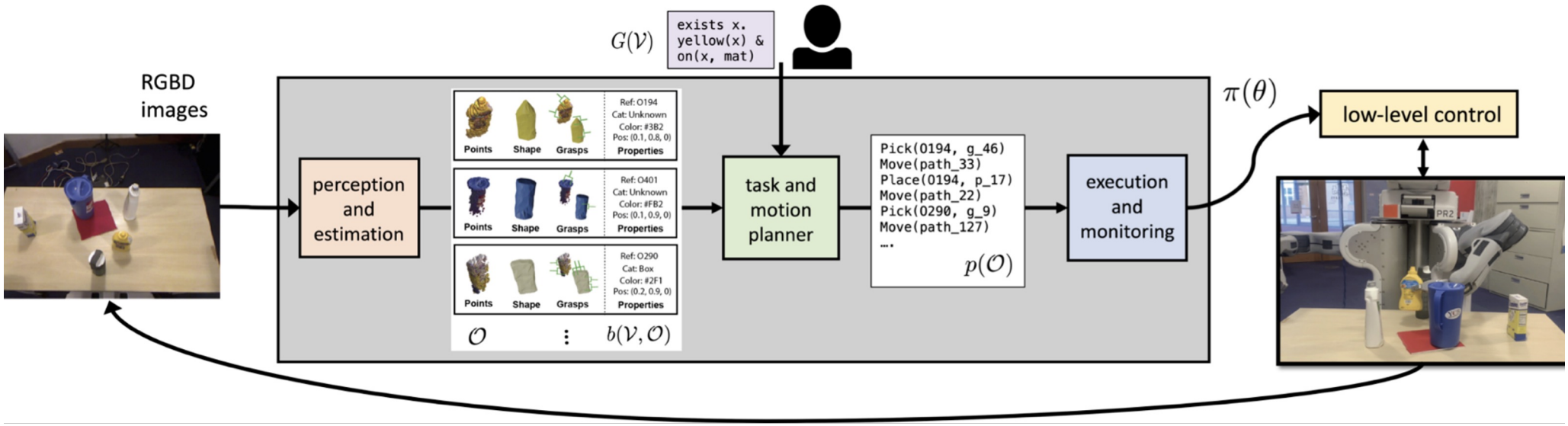
But it can be worthwhile

- Can reduce sample complexity for learning
  [Limits of autoregressive models and their alternatives, Lin et al, 21]
- Can make the easy cases fast!
  - circuit complexity for general-case planning may be terrible
    [What Planning Problems Can A Relational Neural Network Solve?, Mao, et al, NeurIPS23]
- A general goal conditioned policy implicitly encodes a model
  [General Agents Need World Models, Richens, Abel, Bellot, Everitt]

Not hard to "amortize"/"compile" the common cases : train a **partial** policy while you sleep



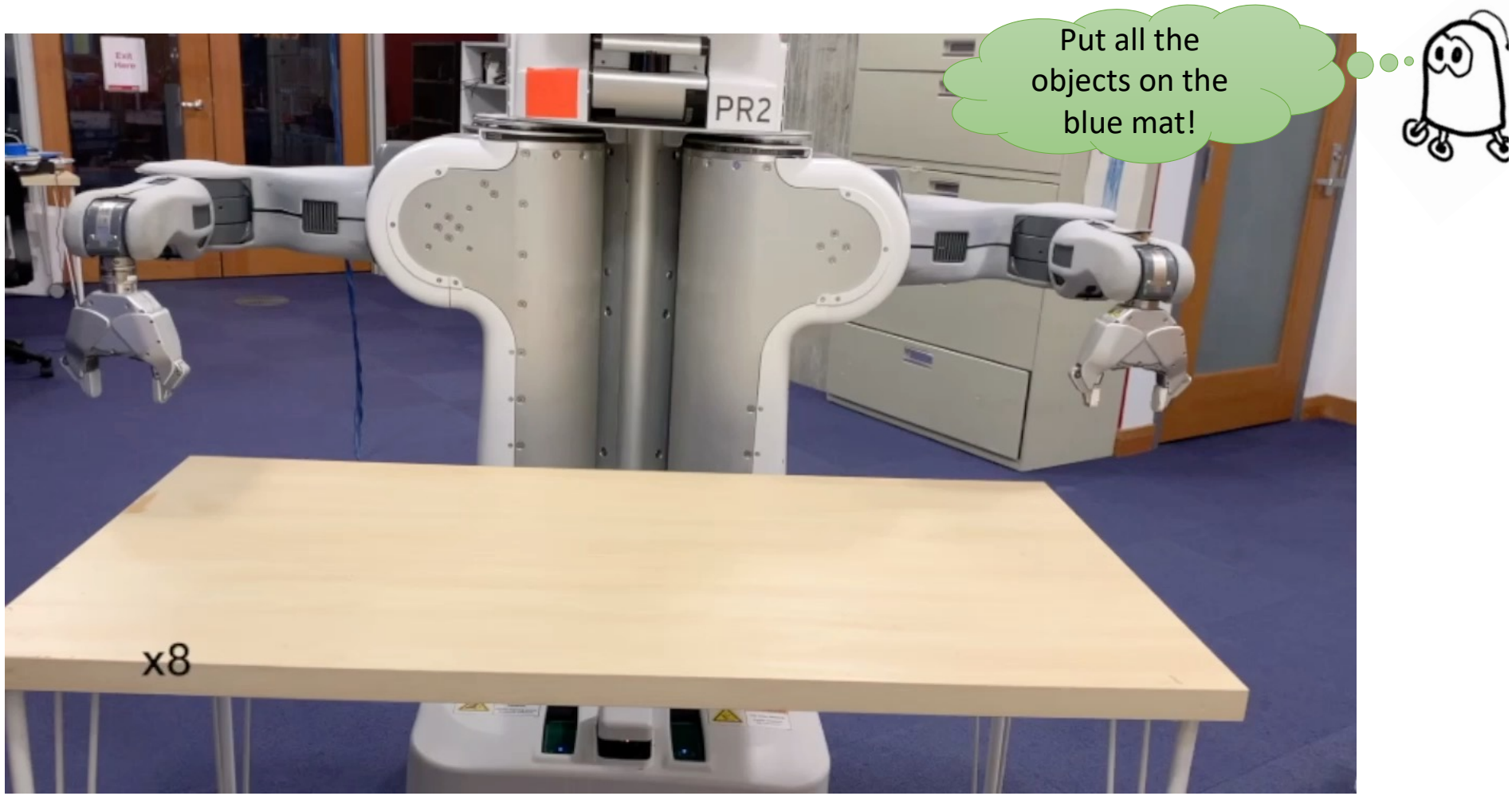
Learned perception plus planning yields generalized policy



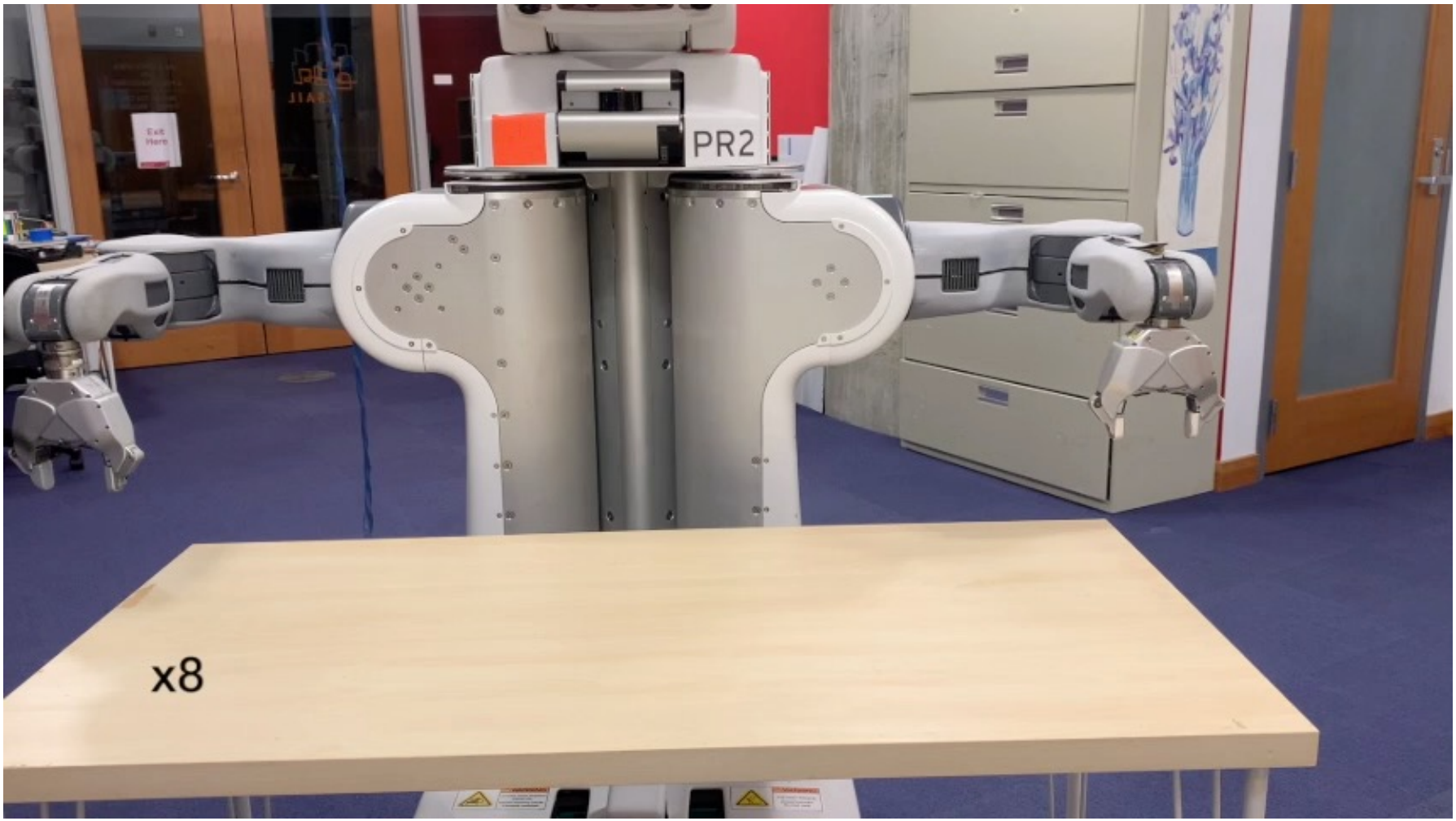


Aidan Curtis, Xiaolin Fang, ..., Caelan Garrett, ICRA 2022

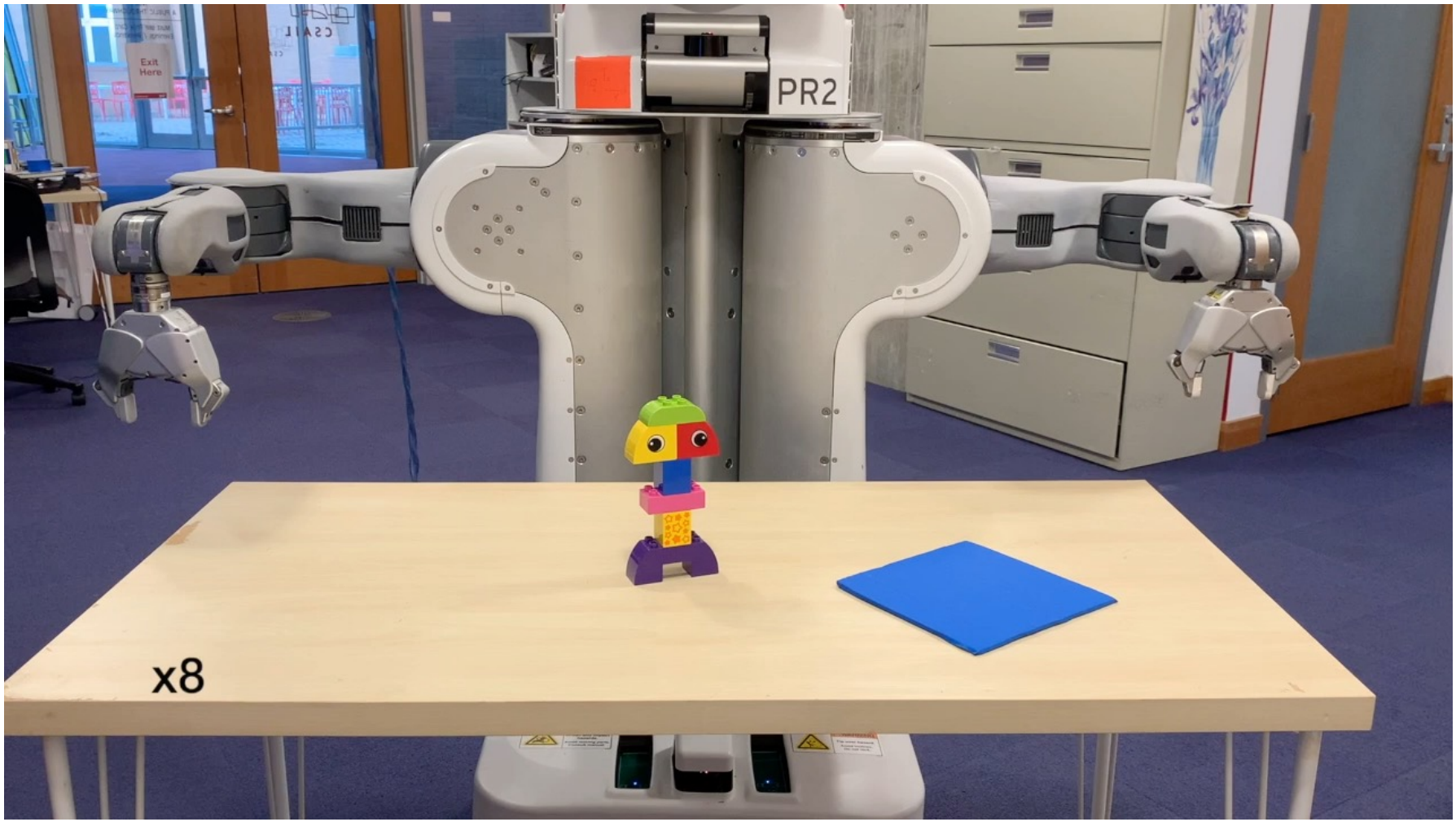
Immediate generalization to novel objects, goals, arrangements



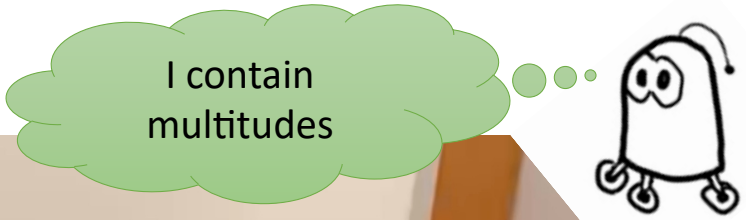
Put each object in the bowl of the closest color

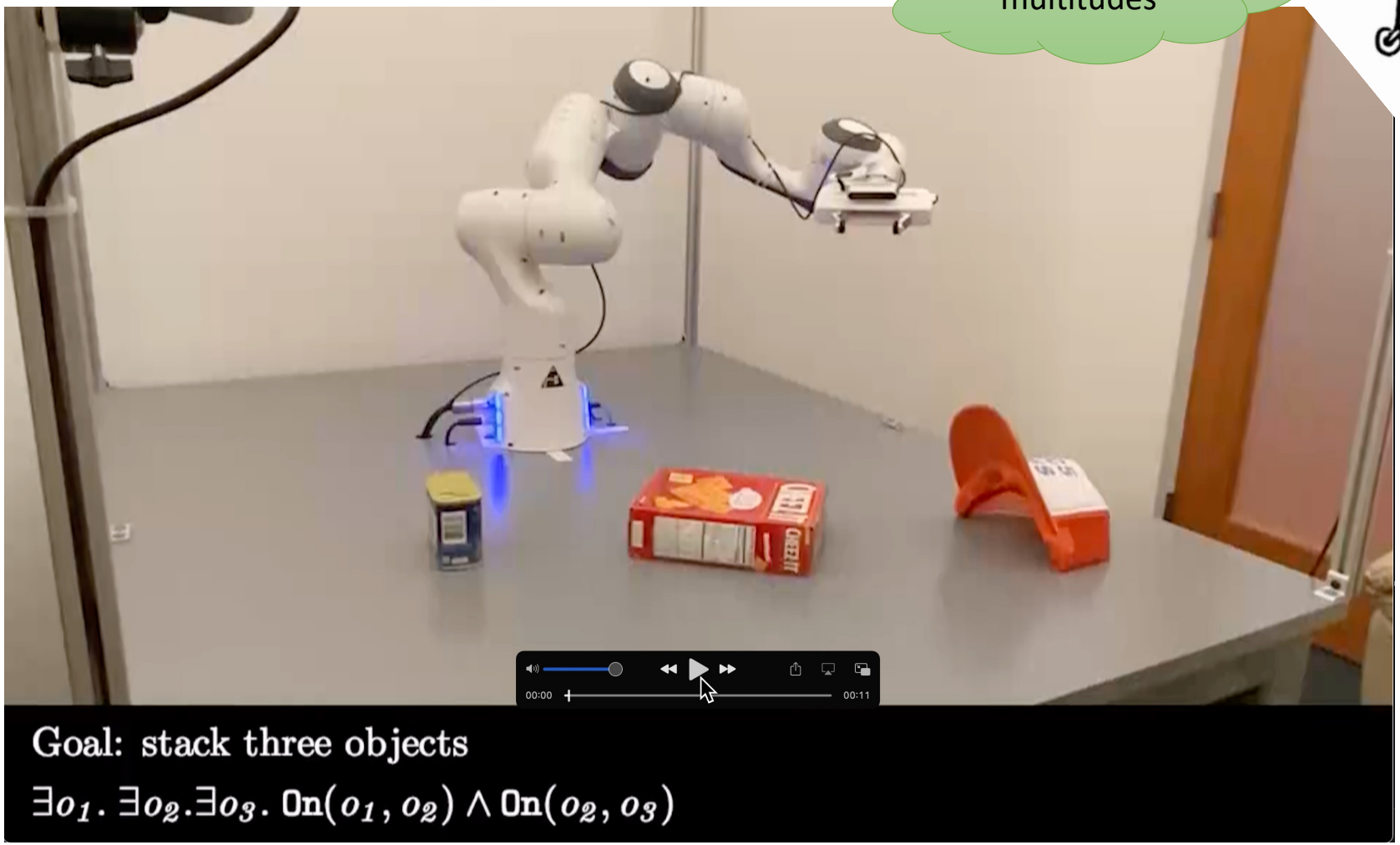


Put all the objects on the blue mat



Cross-embodiment generalization

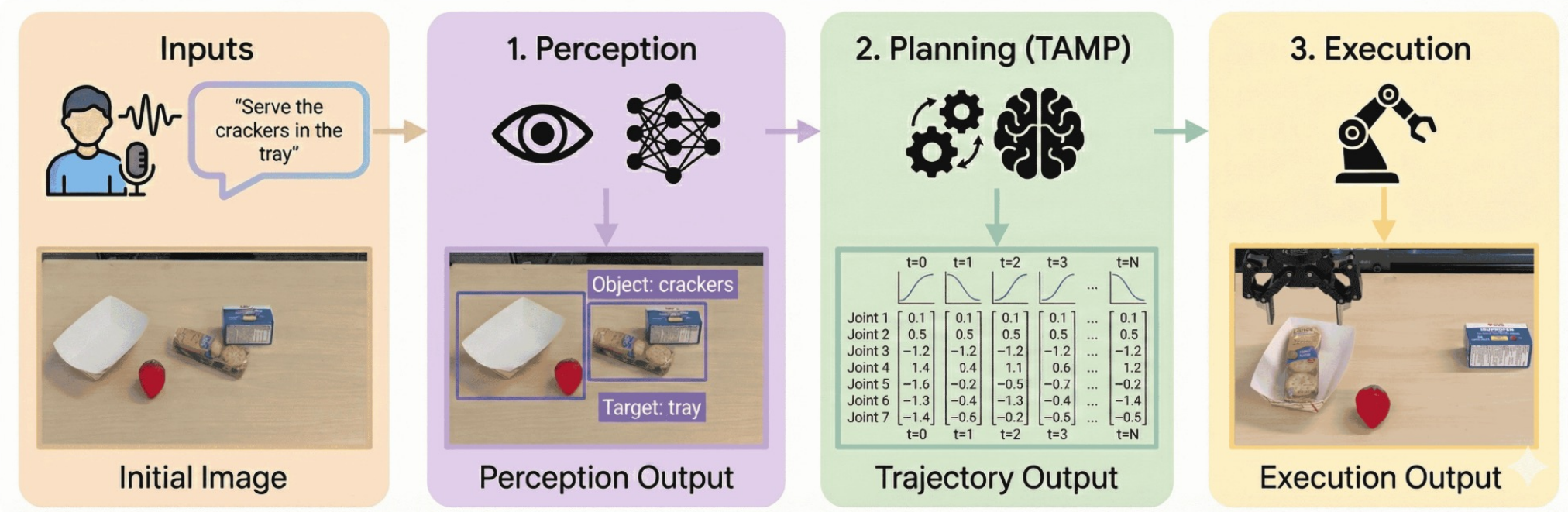




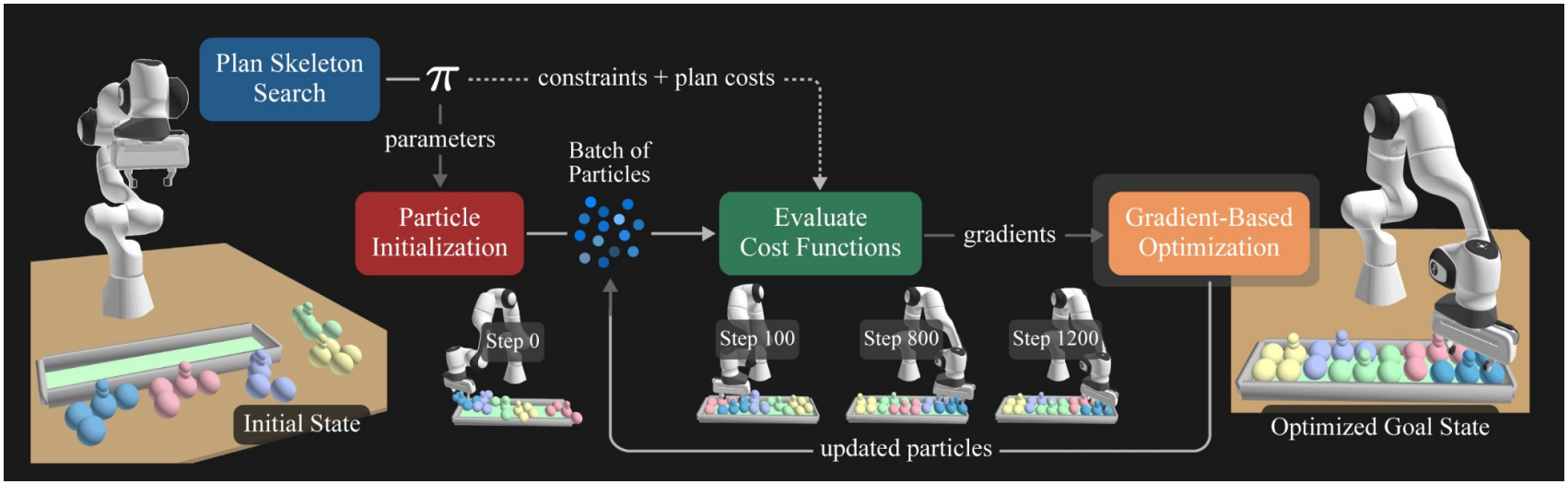
TiPToP [Shen, Kumar, Chintalapudi, Kaelbling, Lozano-Perez 2025]



TiPToP Overview

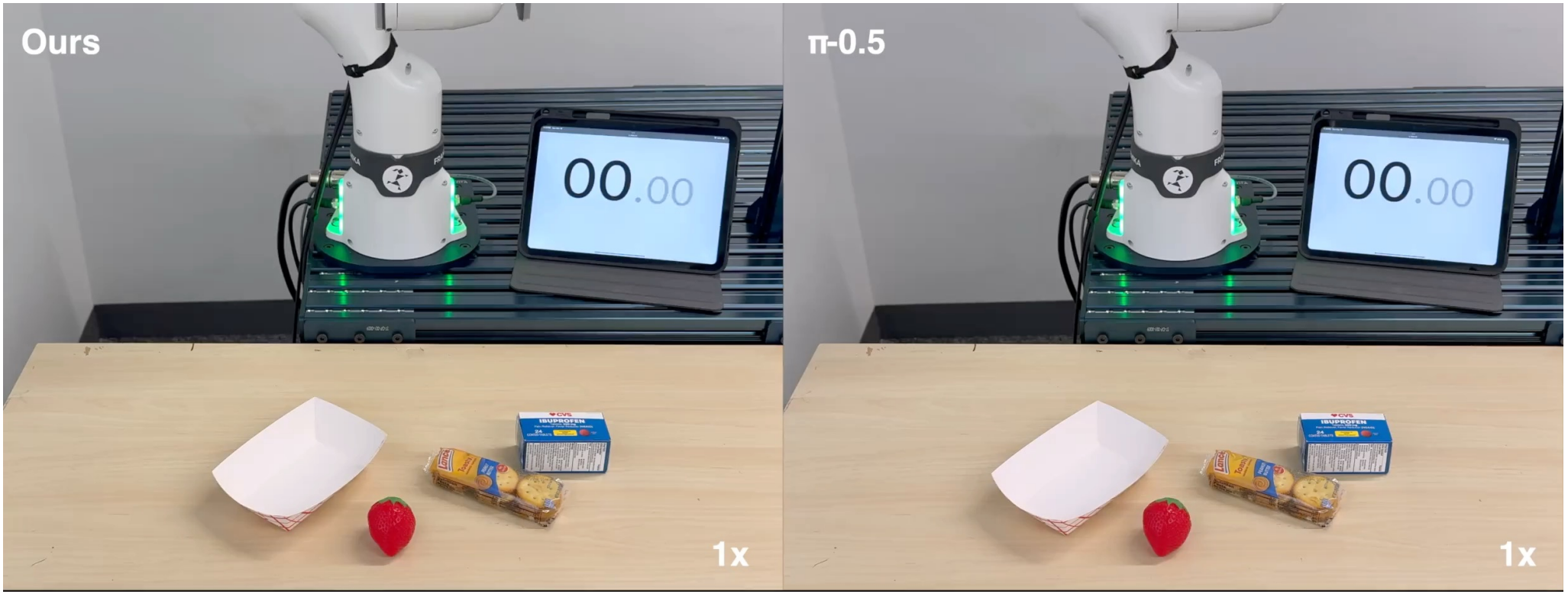


TiPToP is built on cuTAMP



- cuTAMP frames TAMP as a backtracking bilevel search over plan skeletons.
- Each skeleton induces a continuous Constraint Satisfaction Problem that defines the structure of a particle (parameters) and cost functions (constraints and plan costs).
- These particles are optimized in parallel by evaluating their costs with differentiable cost functions, allowing gradient-based optimizers to iteratively update them towards satisfying solutions.

TiPToP vs Pi0.5 Droid



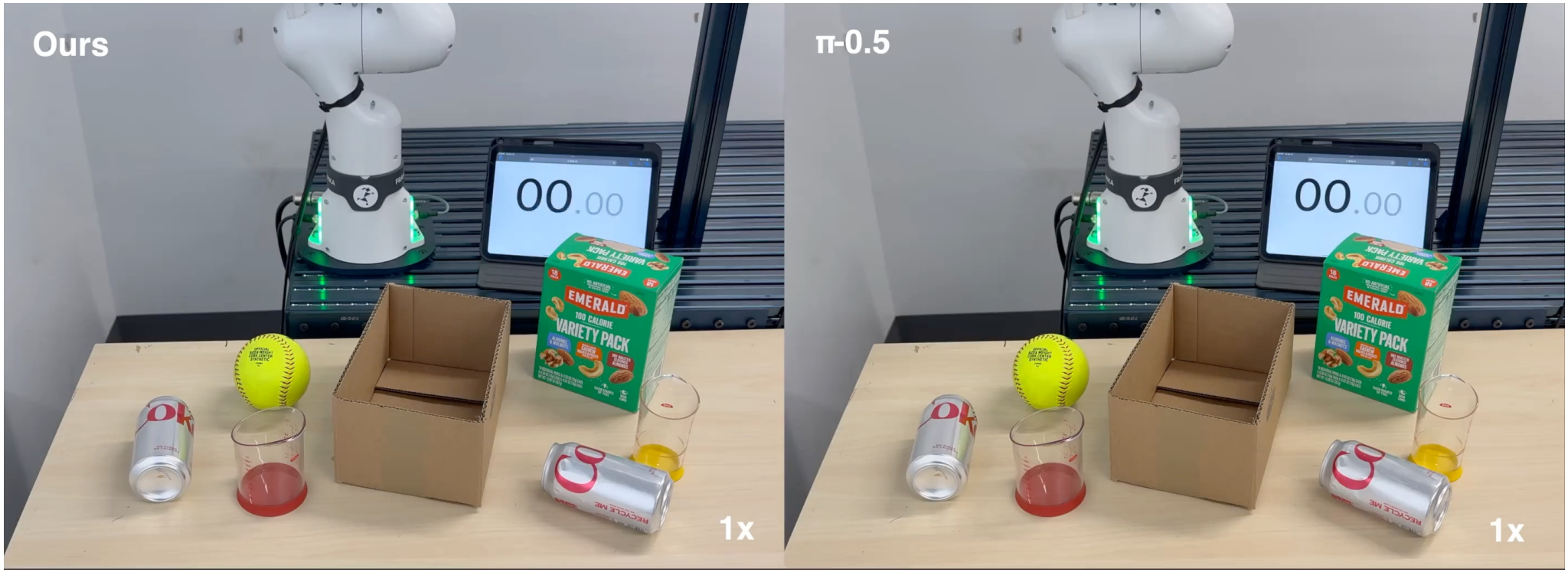
Task instruction: “place the peanut butter crackers onto the tray”

TiPToP vs Pi0.5 Droid



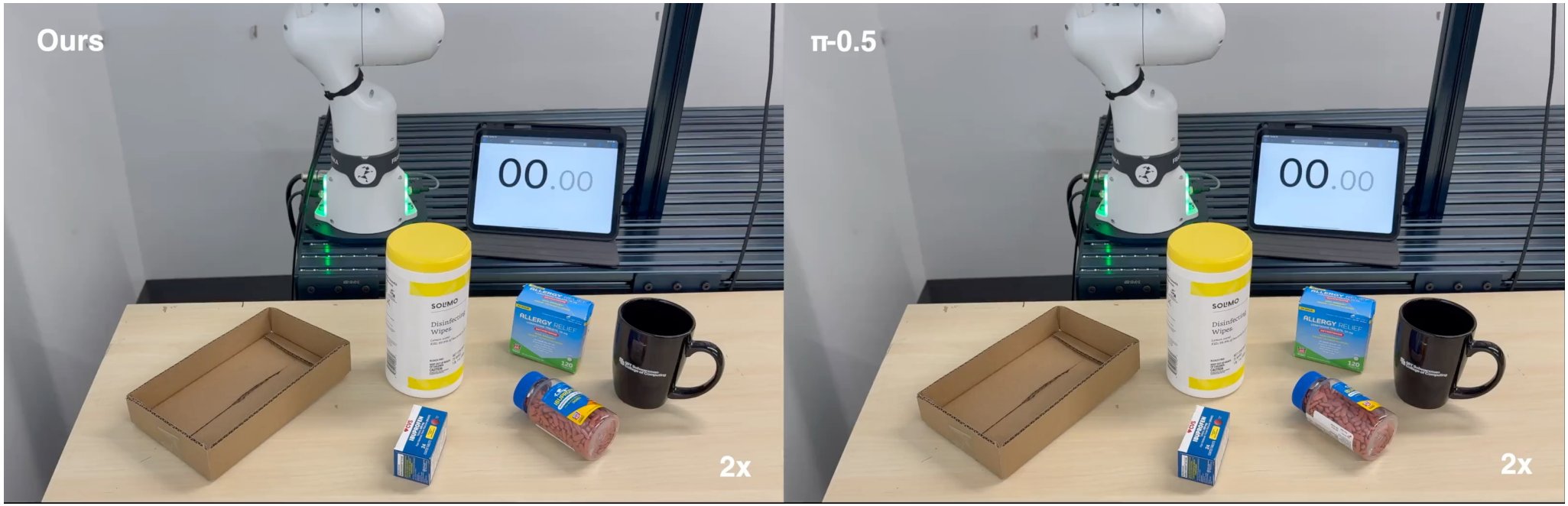
Task instruction: “place the peanut butter crackers onto the tray”

TiPToP vs Pi0.5 Droid



Task instruction: “throw the coke cans into the empty box”

TiPToP vs Pi0.5 Droid

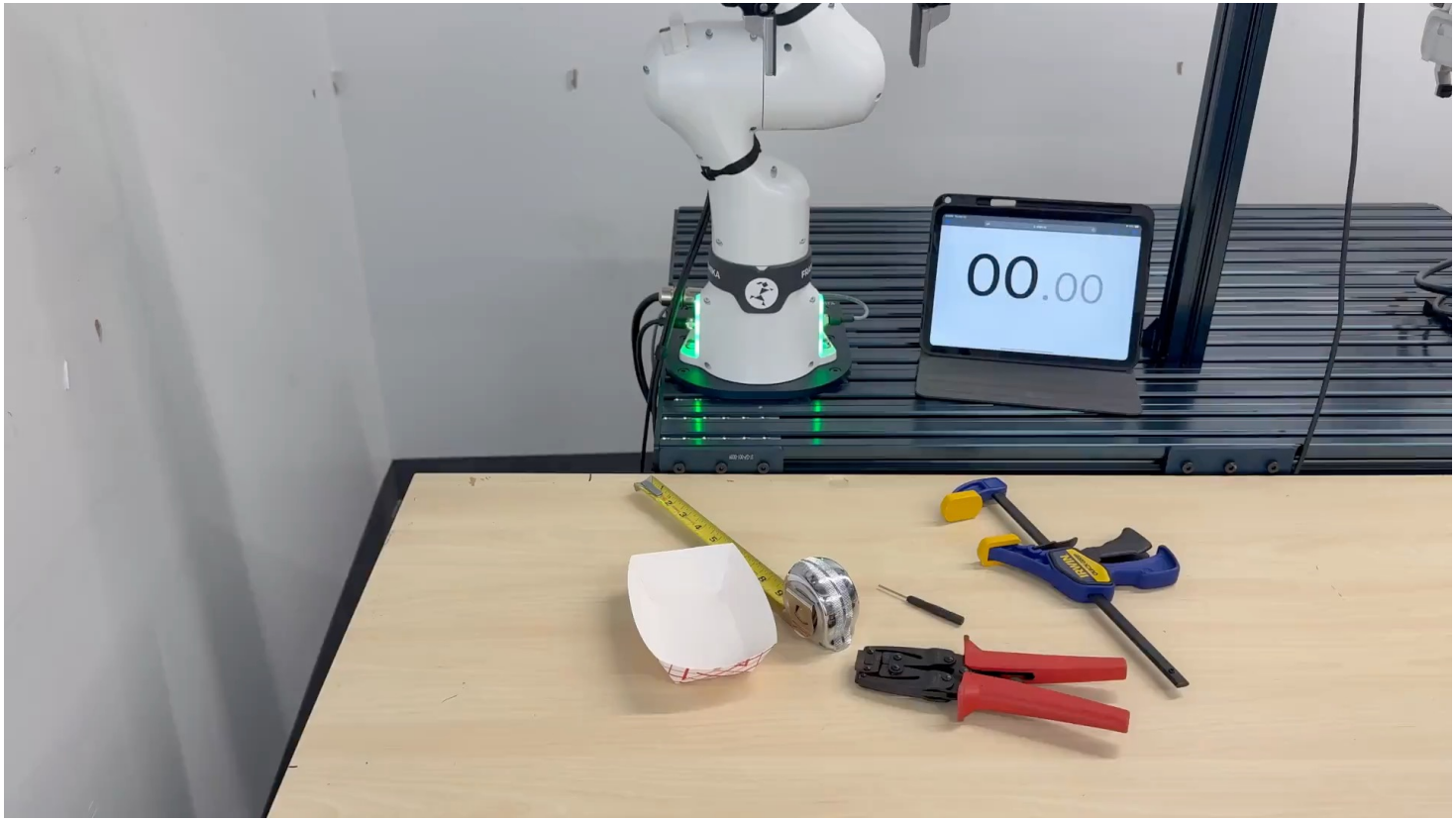


Task instruction: “pack the medicine into the box”

TiPToP Limitations

- Open-loop execution is sufficient for success.
- The underlying TAMP system has the necessary symbols and action primitives to express and solve the task.
- All objects can be approximated well by simple mesh-filling (convex hull) of their segmented point cloud.
- All objects necessary to accomplish the task are fully observable by the wrist camera in the initial state.
- No object reorientation or double-stacking (to be addressed in coming update).

TiPToP Failure due to controller



5 key ideas

1. There is an external physical reality that is worth believing in
2. World-model + planning algorithm = policy
3. Pick representations for learning based on sample efficiency / generalization
4. Learning can make planning better
5. Planning can make learning better

Basis for aggressive generalization: compositionality

In the factory, and during your early life

- learn components
  - procedures
  - language models
  - physics models
  - ....

When faced with a new problem

- make a novel composition of known components
- occasionally add new components

Make infinite use of finite means -- von Humboldt

Make the combinatorial explosion work for us -- lpk

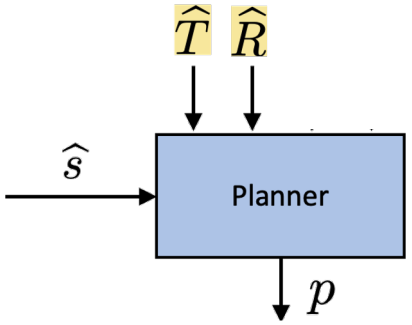


Representing a world model for planning

Transition model T:

- given an **input state** and a choice of **action**
- predict the **next state**

Does not have to be perfectly accurate



Choose a representation that supports

- **sample-efficient learning**: can be learned from relatively small amounts of data
- **incremental learning**: new aspects of T can be learned without interfering with old ones (combine learning in factory and wild)
- **efficient reasoning**: plans can be made reasonably computationally efficiently

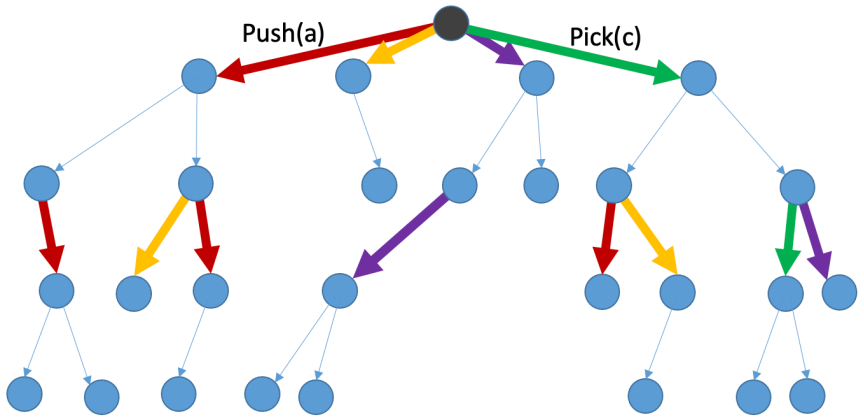


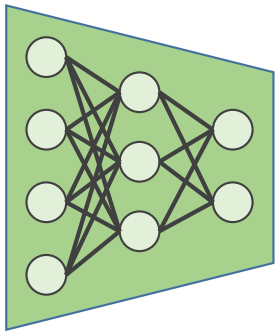
Image to image

Directly use a neural network to map images to images

- Requires complex network
- Difficult to generalize
- Quickly loses precision when applied in sequence

$$T$$



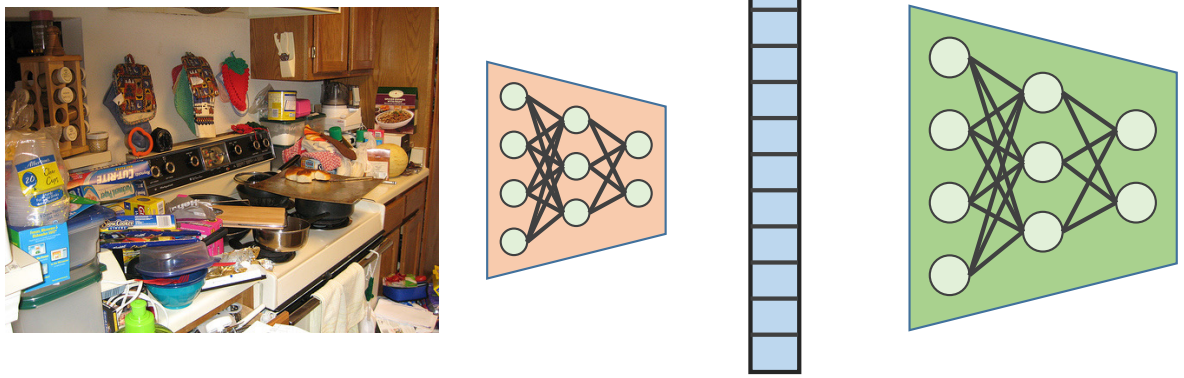




Latent, compressed encoding

Map input state into lower-dimensional vector

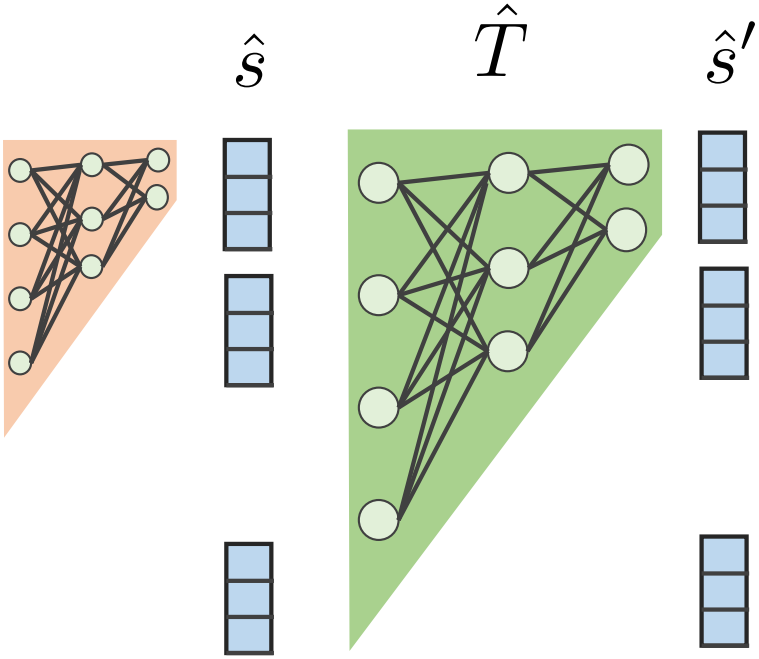
- $\hat{s}$ is an abstraction
- $\hat{T}$ is smaller, easier to learn



~~Factored~~ encoding

disentangled





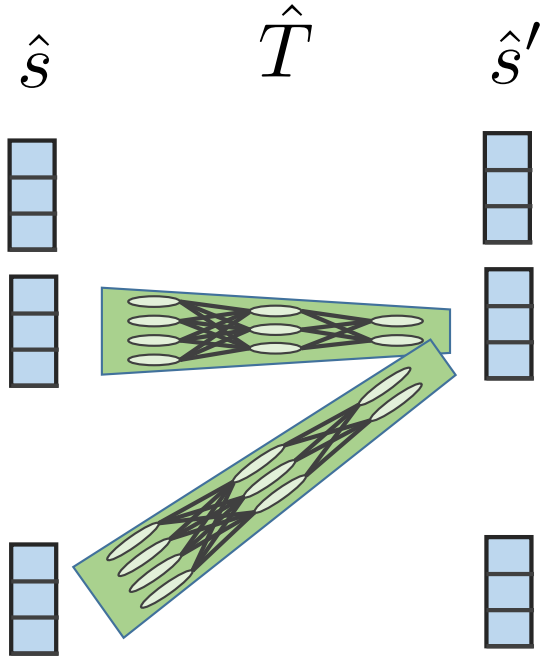
Divide latent vector into individually meaningful pieces called factors

- learn separate transition model for each factor
- not yet very advantageous (but does enable IW() style planning

F1: temperature
F2: heater-running
F3: window-open
F4: precipitation

~~Factored~~ encoding: enables locality and sparsity

disentangled



Locality: whether and how factors change depends on only a few other factors

Sparsity: most factors don't change most of the time

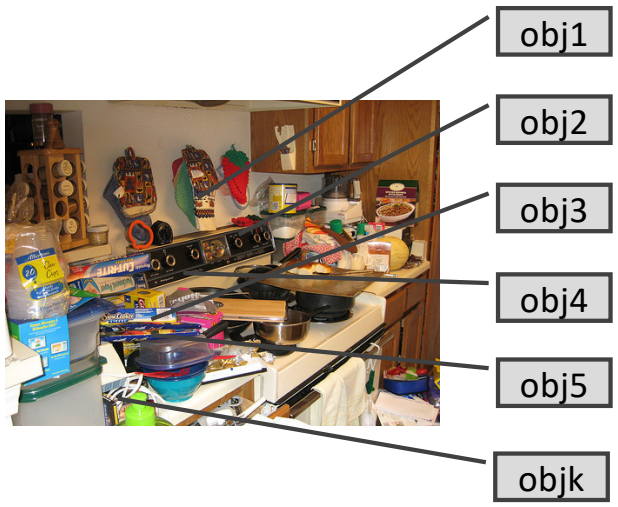
if heater-running:
temperature := f(temperature, window-opening)

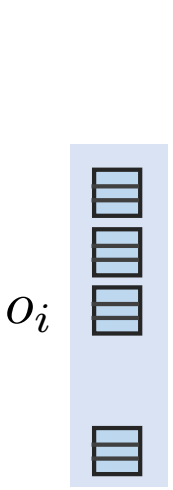
Can result in **dramatic reduction in complexity** of the function to be learned

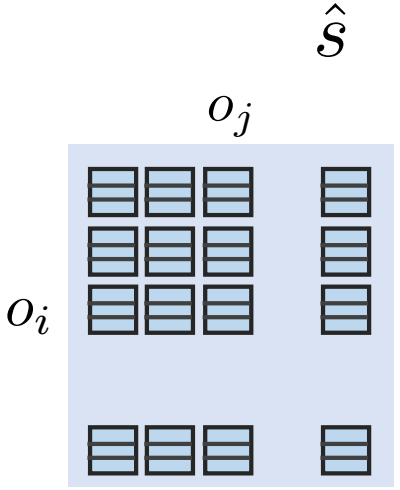
Enables **powerful general-purpose search "heuristics"** based on an assumption that actions don't "interfere" with each other

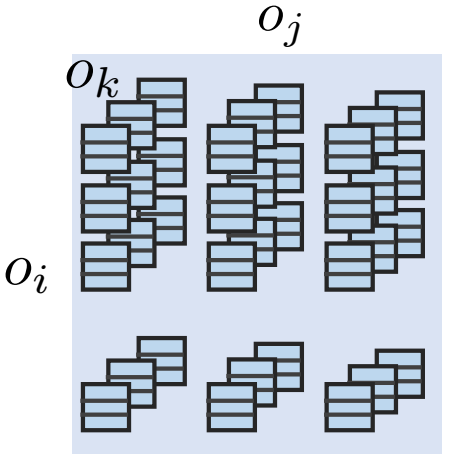
~~"Lifted" (object-based)~~ encoding: objects index factors

slot-based









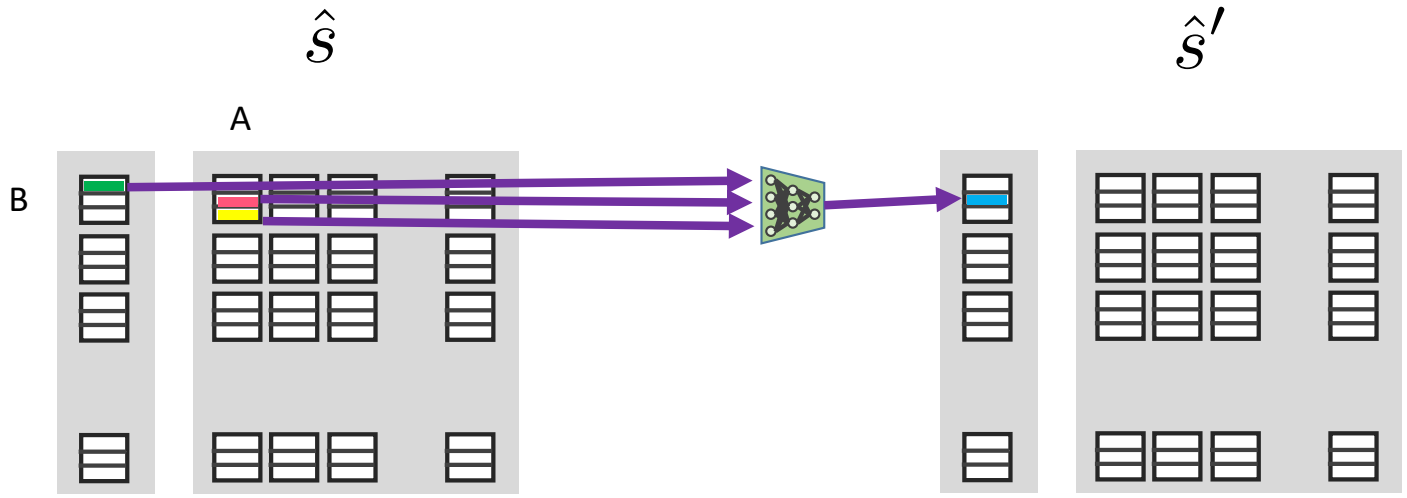
several factors
for each object:
properties

$temp(o_i)$
$open(o_i)$

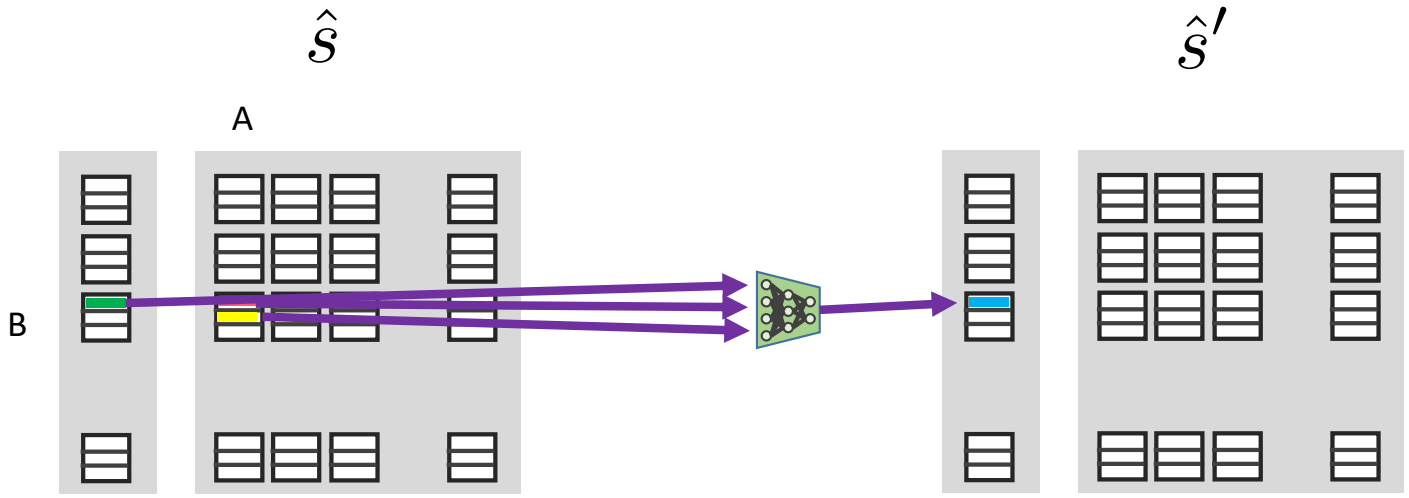
several factors for each
tuple of objects:
relations

$relpose(o_i, o_j)$
$contains(o_i, o_j)$

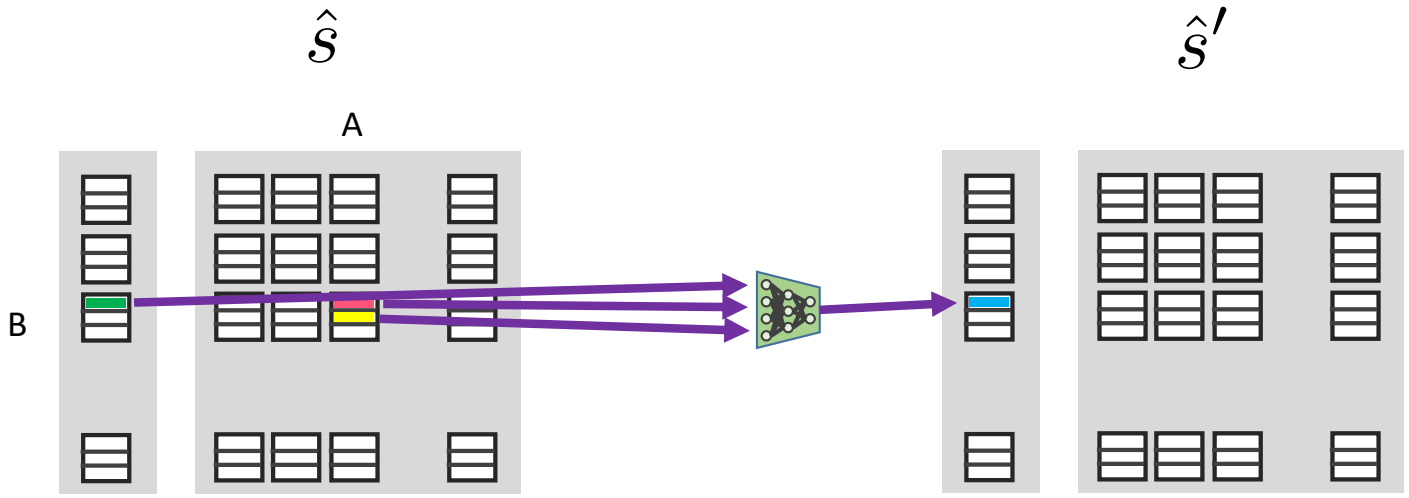
Sparse, factored, lifted transition rules



Sparse, factored, lifted transition rules



Sparse, factored, lifted transition rules



Causal Action Model: lifted, factored, transition model

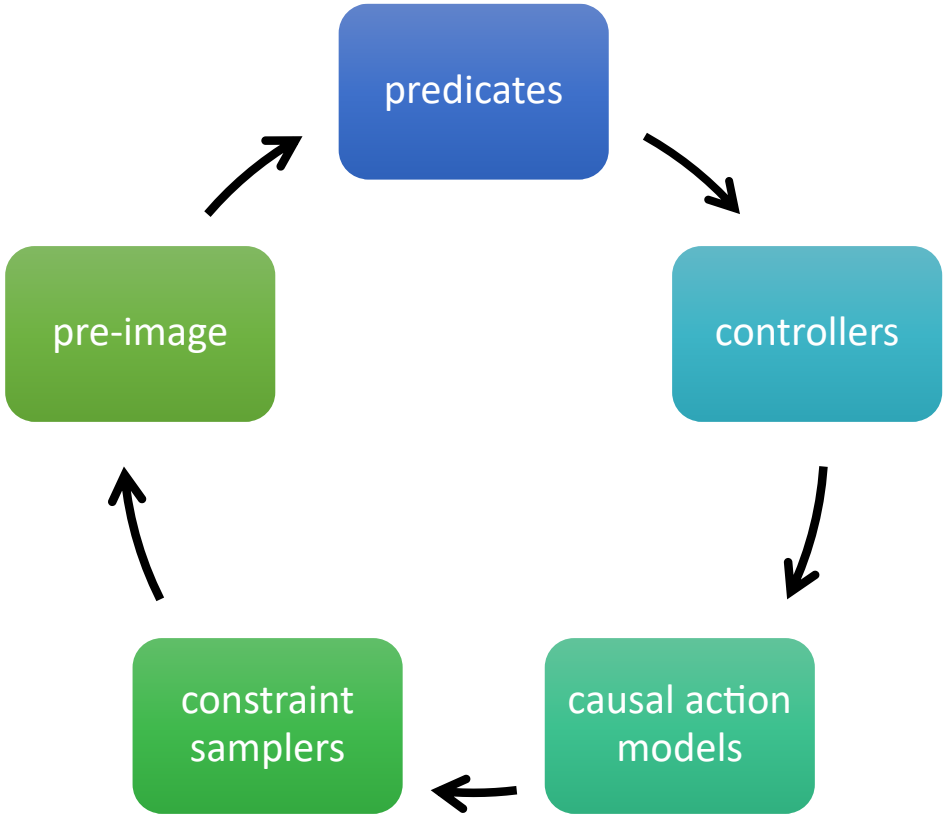
```
behavior b1(X) :
  body:
    bind d: value_generator_1
    unordered:
       achieve f4(X) == False
       achieve f2() == g(X, d)
    do controller_3(d)
  effect:
    f2() == X
```

Interpretation

- For any value of d
- If the **achieve** conditions are true
- And we **execute** the controller
- The **effects** will be true afterwards

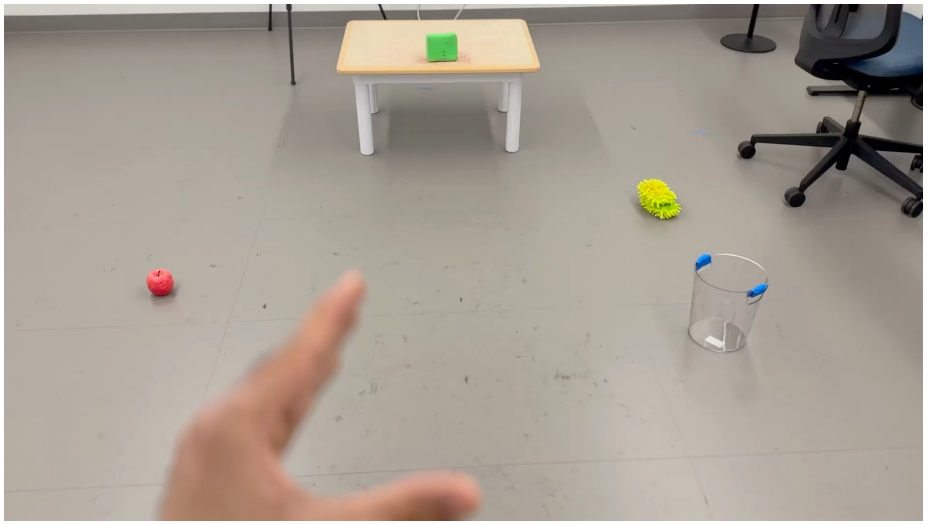
Jiayuan Mao, Crow Description Language, Hybrid Representations for Decision-Making, WAFR24

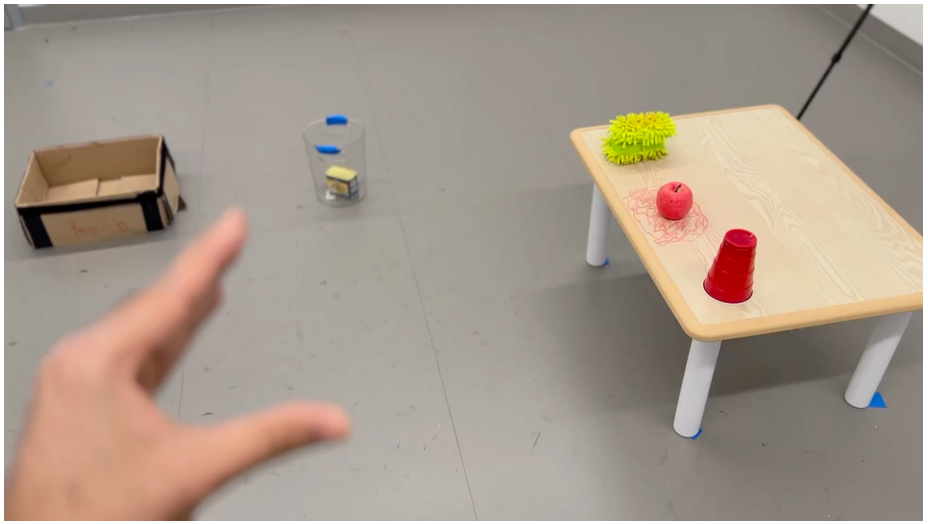
Continual Learning for Causal Action Models



- Start with some **built-in predicates** (innate or via LVM or teaching)
- **Learn controllers** to achieve them (RL on top of sensible control-theoretic primitives)
- **Instantiate CAMs** with the predicate as effect and controller
- Learn, by experimentation, **generative models for sampling** continuous parameters
- **Describe the preimage** of the effect under the controller (sort of like but not the same as an initiation set) using existing predicates, if possible, but if not, instantiate **new predicates**
- ....

Given a set of skills, we can learn CAMs from a small number of demonstrations

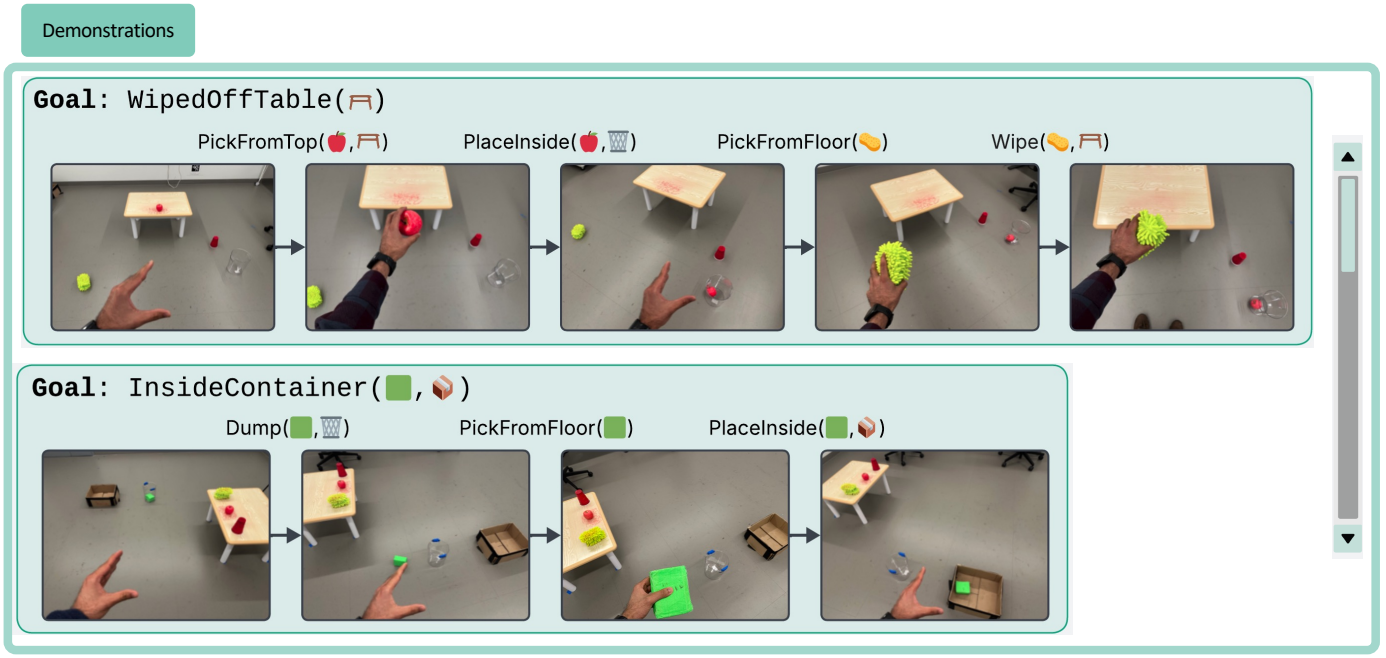


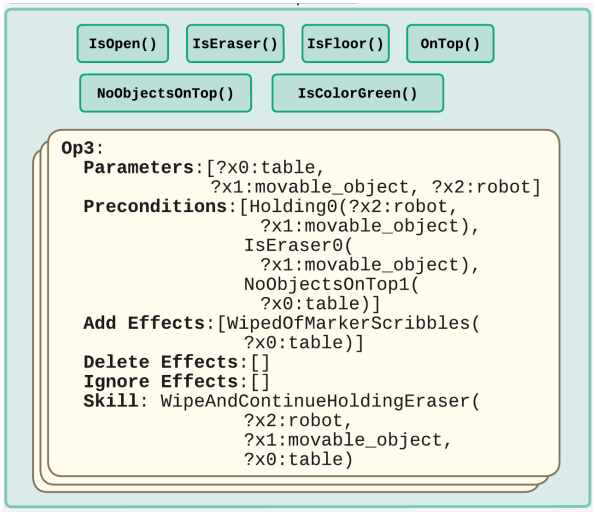


...

Silver, Chitnis, McClinton, Kumar, ... AAAI 2023

Demos have controllers and final goal labeled





Extreme generalization to new situations via planning

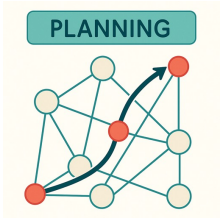
Causal Action Model

IsOpen() IsEraser() IsFloor() OnTop()

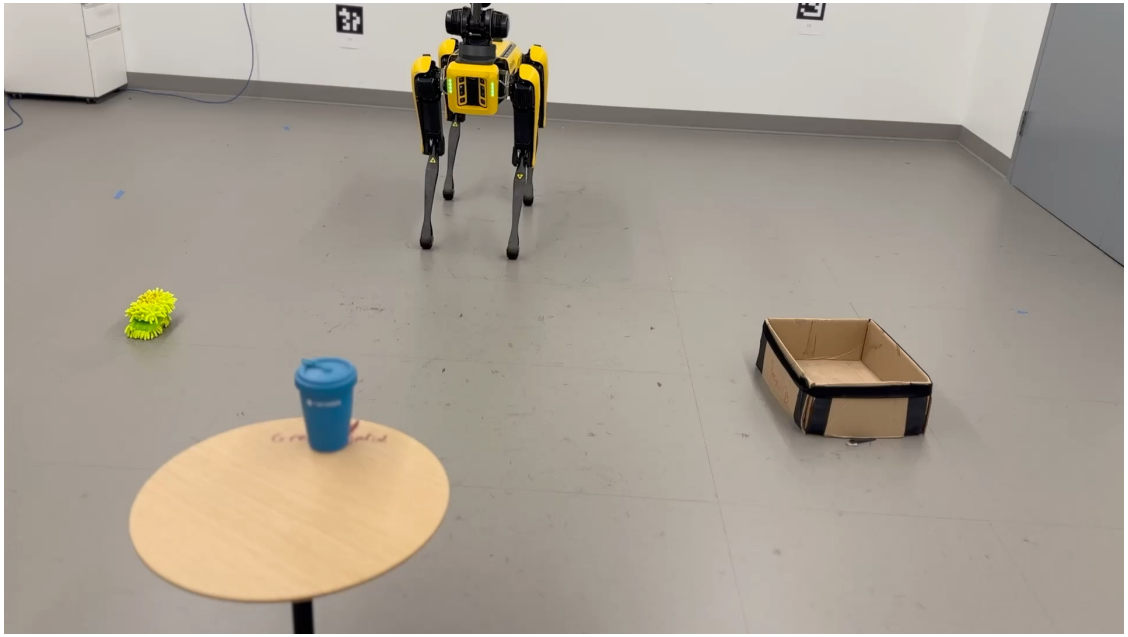
NoObjectsOnTop() IsColorGreen()

```
Op3:
  Parameters:[?x0:table,
              ?x1:movable_object, ?x2:robot]
  Preconditions:[Holding0(?x2:robot,
                  ?x1:movable_object),
                 IsEraser0(
                  ?x1:movable_object),
                 NoObjectsOnTop1(
                  ?x0:table)]
  Add Effects:[WipedOfMarkerScribbles(
                 ?x0:table)]
  Delete Effects:[]
  Ignore Effects:[]
  Skill: WipeAndContinueHoldingEraser(
                 ?x2:robot,
                 ?x1:movable_object,
                 ?x0:table)
```

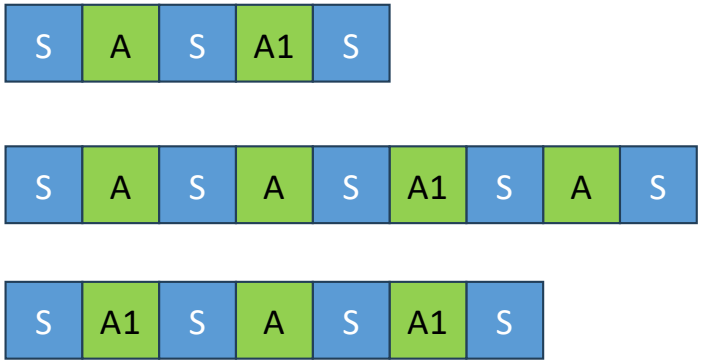
+



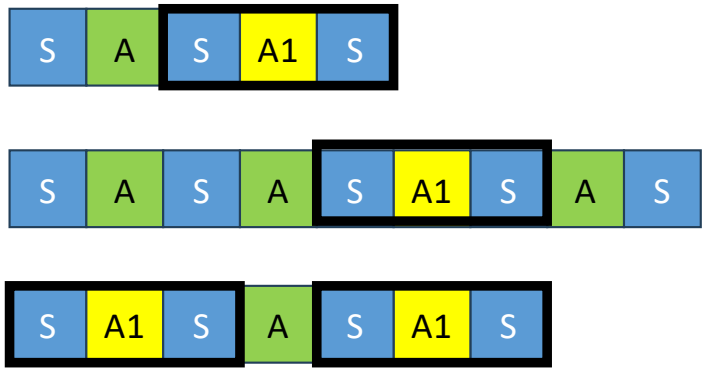
Goal: WipedOffTable(🟤)



If we knew skills and a good set of predicates, we could learn CAMs



If we knew a good set of predicates, we could learn CAMs



s

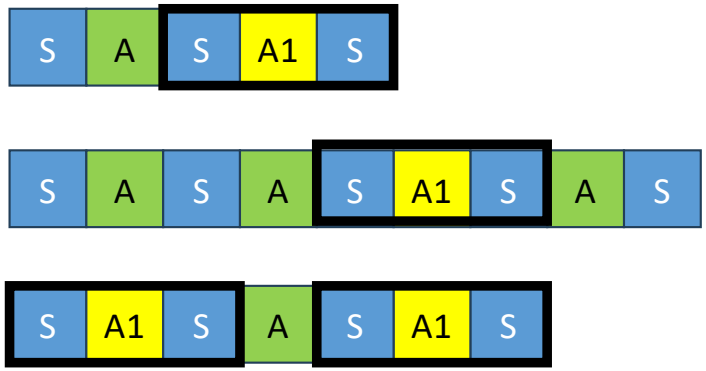
empty(x1)
resting(x2, x5)
contained(x3,x1)
on_front(x4,x5)
oven(x5)
button(x4)
on_front(x6,x5)
light(x6)
dim(x6)
cool(x5)

push(x4,3.2)

s'

empty(x1)
resting(x2, x5)
contained(x3,x1)
on_front(x4,x5)
oven(x5)
button(x4)
on_front(x6,x5)
light(x6)
bright(x6)
warm(x5)

If we knew a good set of predicates, we could learn CAMs



Just look at the changes!

s

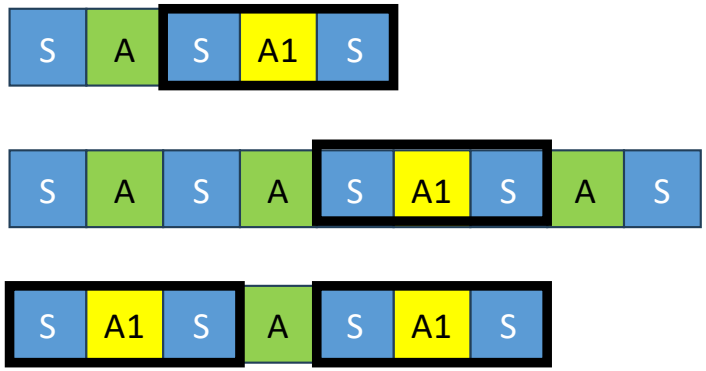
empty(x1)
unscrewed(x2)
contained(x3,x1)
on_front(x4,x5)
oven(x5)
button(x4)
on_front(x6,x5)
light(x6)
dim(x6)
cool(x5)

push(x4,3.2)

s'

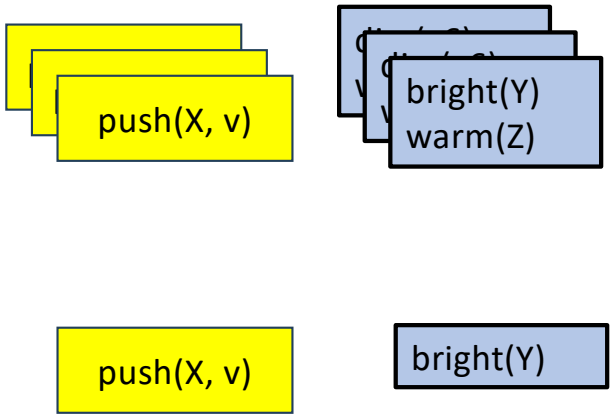
empty(x1)
unscrewed(x2)
contained(x3,x1)
on_front(x4,x5)
oven(x5)
button(x4)
on_front(x6,x5)
light(x6)
bright(x6)
warm(x5)

If we knew a good set of predicates, we could learn CAMs

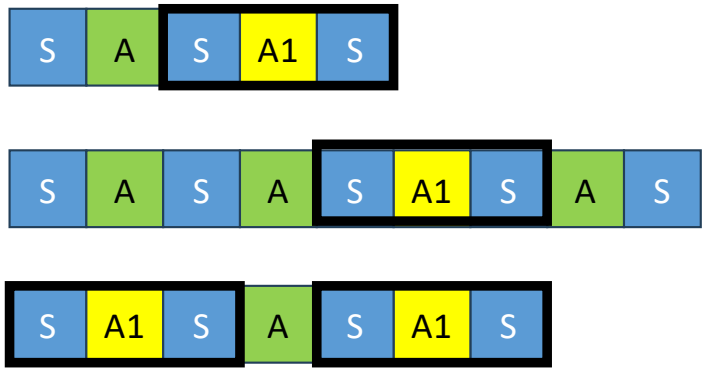


Group transitions with same action and effects

Turn constants to variables



If we knew a good set of predicates, we could learn CAMs



Find conditions about relevant objects that were true in all cases

Symbolic part of CAM

preconds

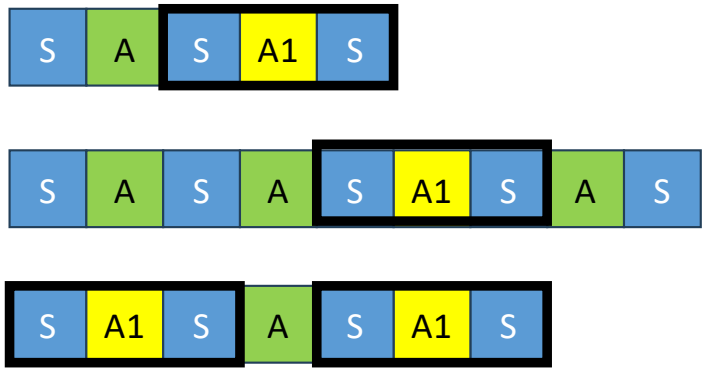
on_front(X,Z)
oven(Z)
button(X)
on_front(Y,Z)
light(Y)
dim(Y)
cool(Z)

push(X, v)

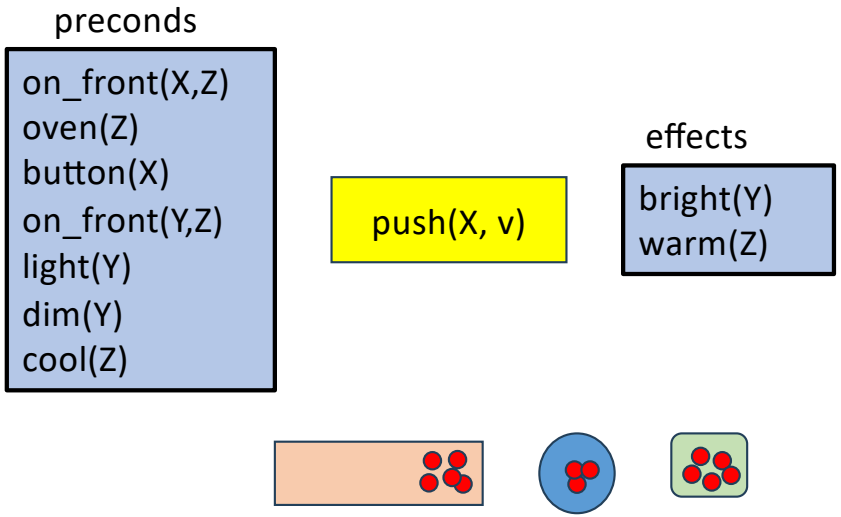
effects

bright(Y)
warm(Z)

If we knew a good set of predicates, we could learn CAMs

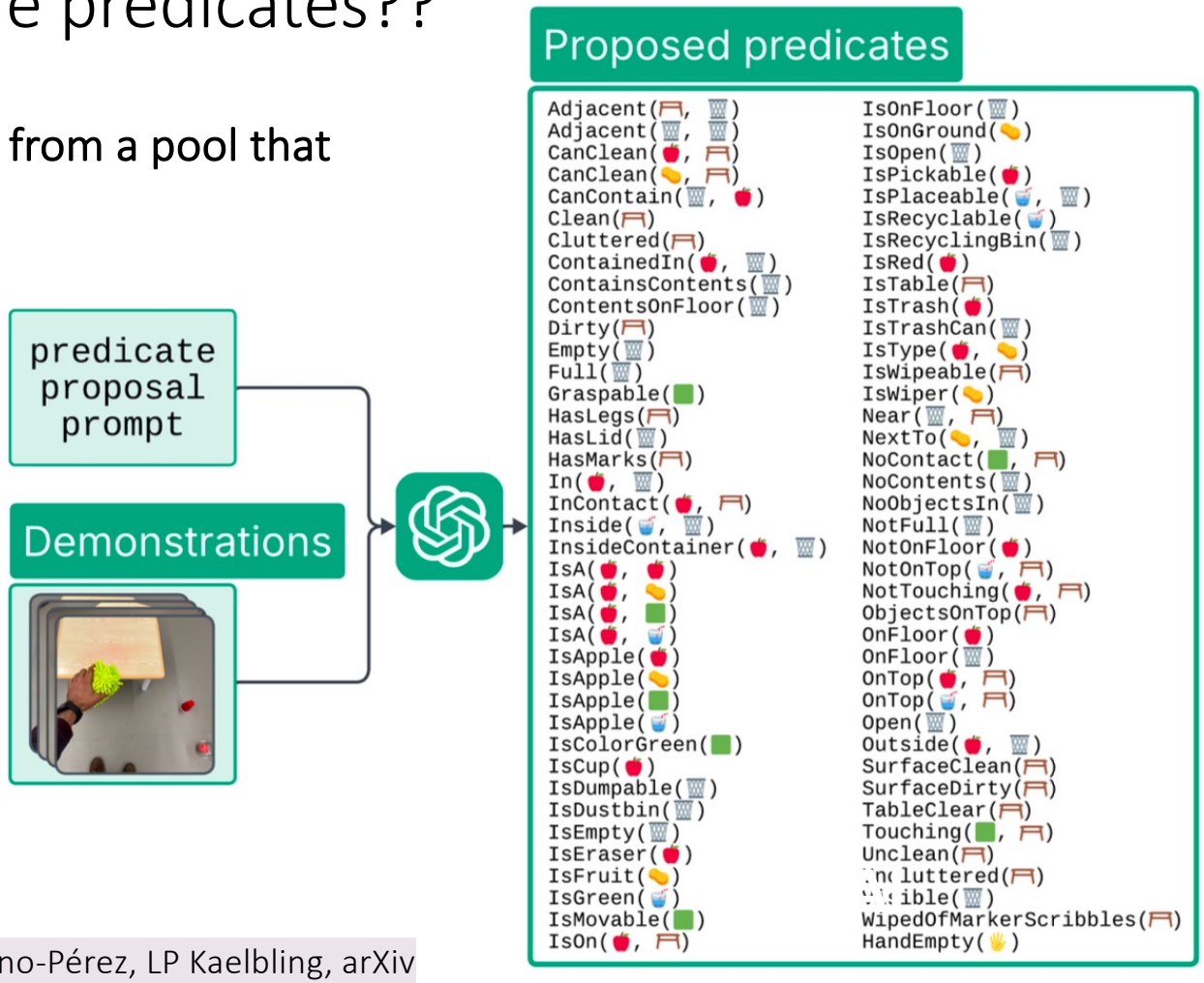


Finally, learn sampler – generative model – for v as a function of properties of X



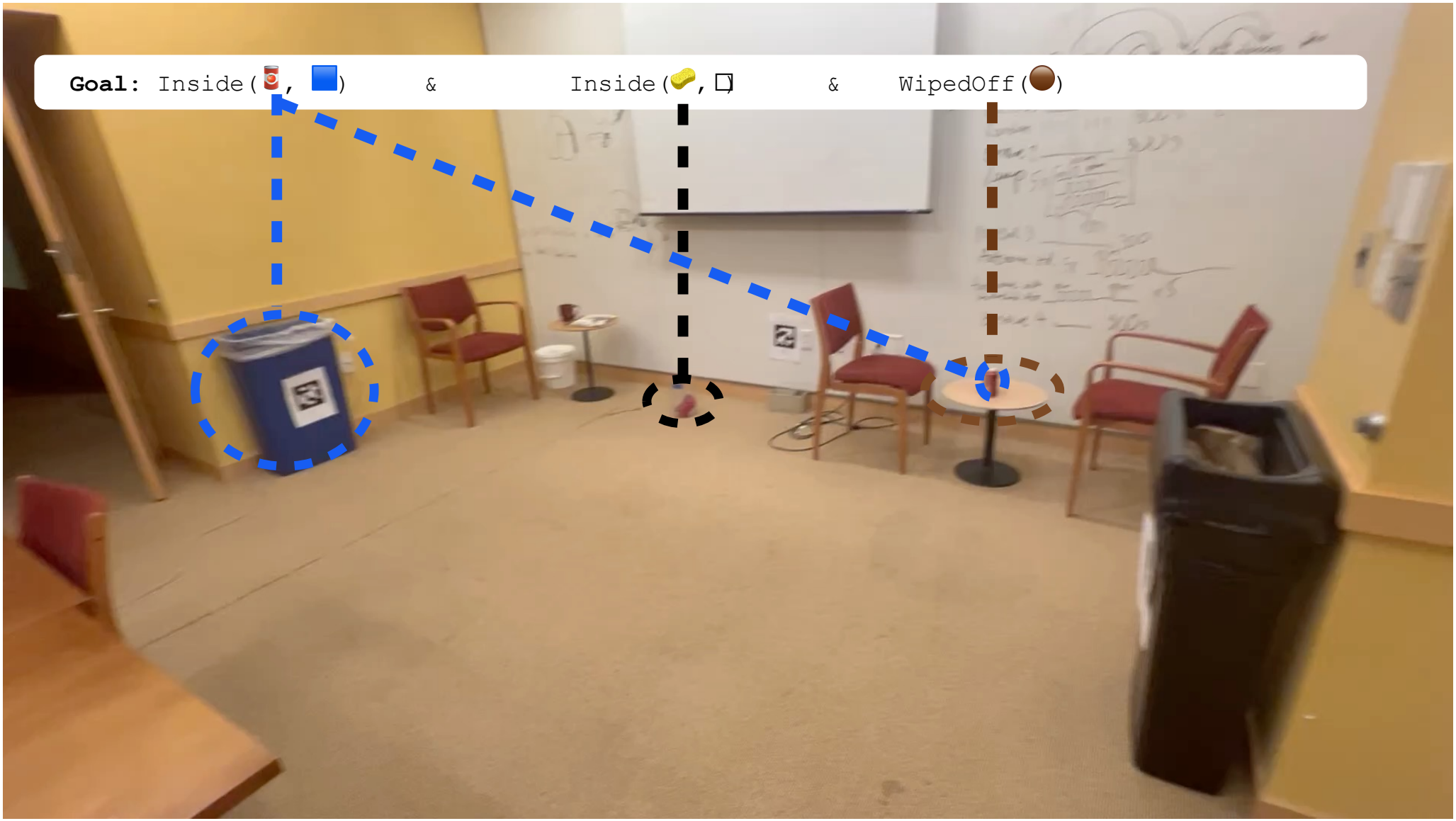
But where do we get the predicates??

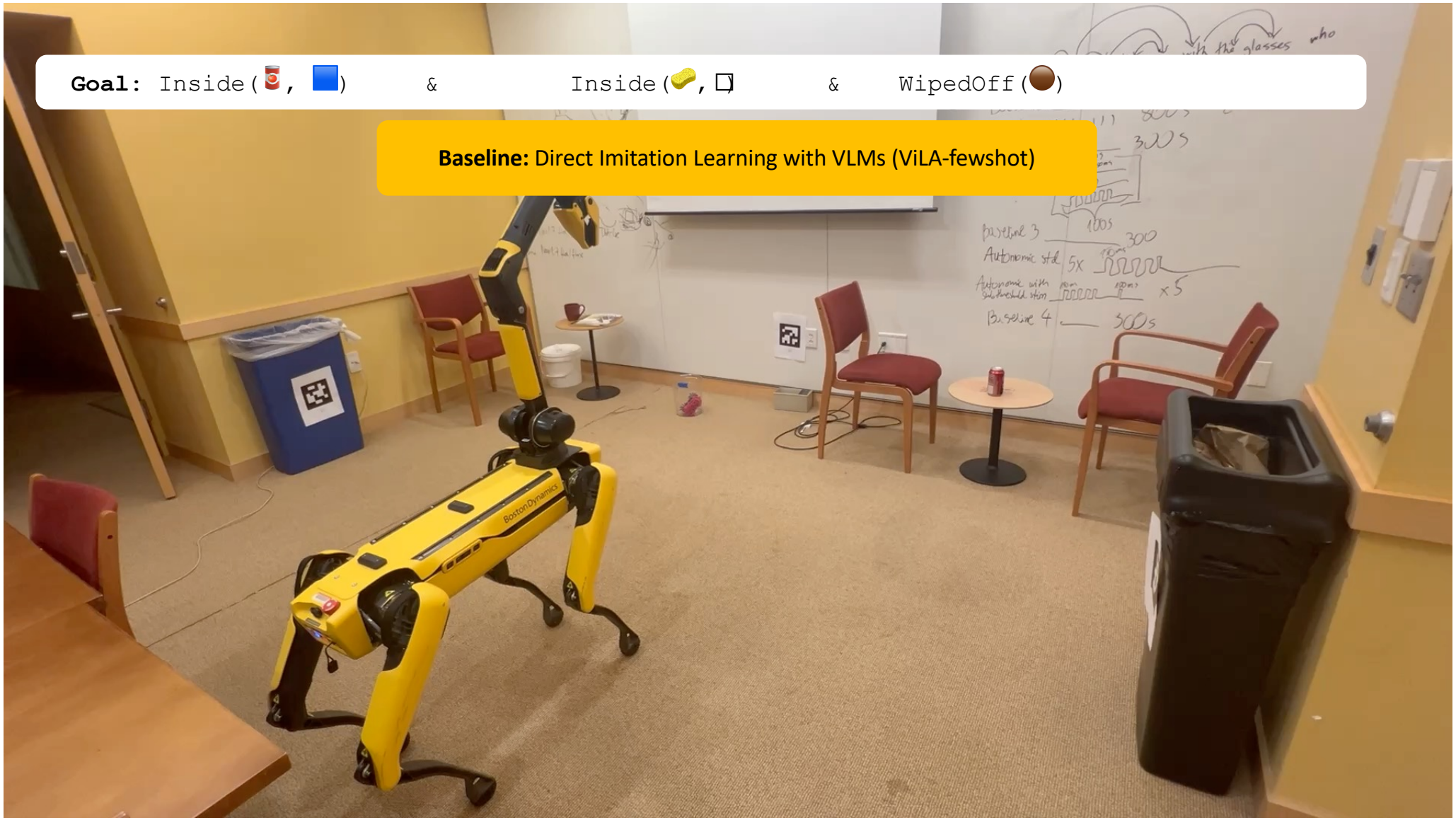
- Hill-climbing search selects subset from a pool that generate high-quality set of CAMs
- Where do we get the pool?
  A VLM!
  - Given the demonstrations, prompt VLM to suggest possible predicates
  - Given an image
    - Run a segmentation alg
    - Ask VLM to label truth values of all predicate instantiations

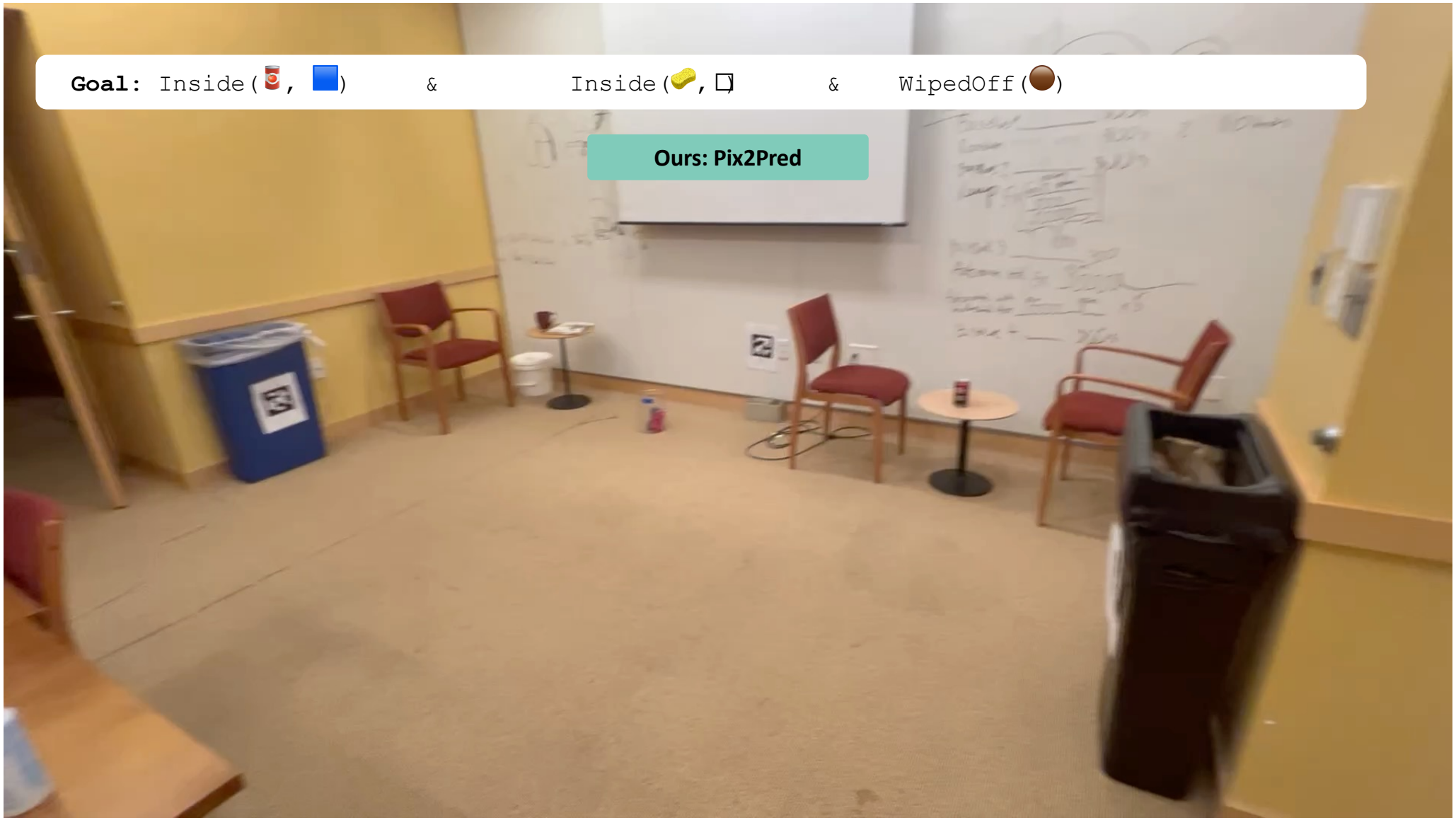


A Athalye, N Kumar, T Silver, Y Liang, T Lozano-Pérez, LP Kaelbling, arXiv

We learned from **6** demonstrations and transfer **zero-shot** to solving a novel goal in a novel environment.

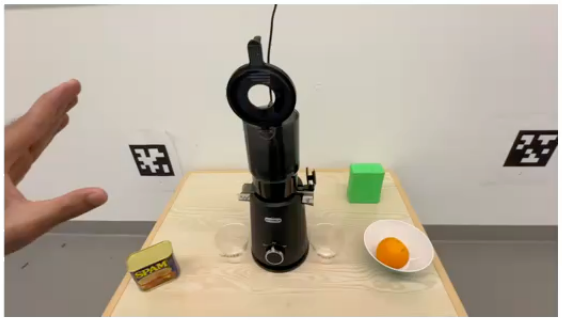

Goal: Inside(,) & Inside(,) & WipedOff( )


Goal: Inside(,) & Inside(,) & WipedOff( )
Baseline: Direct Imitation Learning with VLMs (ViLA-fewshot)
Boston Dynamics


Goal: Inside(,) & Inside(,) & WipedOff( )
Ours: Pix2Pred

Training data for operating a juicer

Initial State



Pick(🍊)

Juicy results from predicate and CAM learning

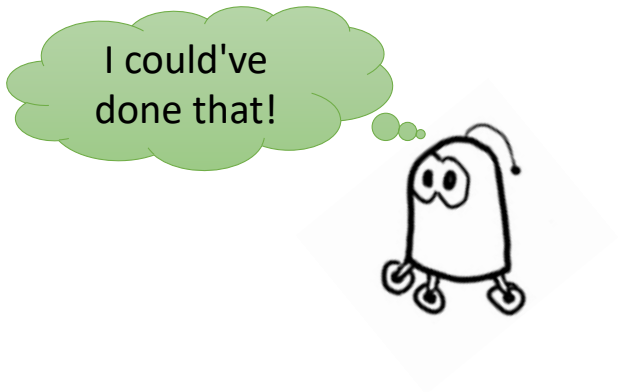
```
Op10(?x0, ?x1, ?x2, ?x3, ? x4):
Preconditions :
    Container(?x0), Container(?x1), Robot(?x2), Juicer(?x3), MovableObj(?x4),
    Empty(?x0), Empty(?x1), Functional0(?x3),
    IsCup0(?x0), IsCup0(?x1), IsPlastic0(?x1),
    InsideWasteValveRegion0(?x1, ?x3),
    NearJuiceValve0(x0, ?x3)
    HandEmpty0(?x2), Inside(?x4, ?x3),
    LidClosed0(?x3),
Add Effects :
    HasContents0(?x0), HasContents0(?x1),
    JuiceInCup(?x4, ?x0), OutsideJuiceMachine0(?x4)
Delete Effects :
    Empty(?x0), Empty(?x1), Inside(?x4, ?x3)
Skill :
    TurnOnAndRunMachine(?x2, ?x3, ?x0, ?x1)
```


BUTCHER
BAKER
FARMER
ROASTER
Boston Dynamics

5 key ideas

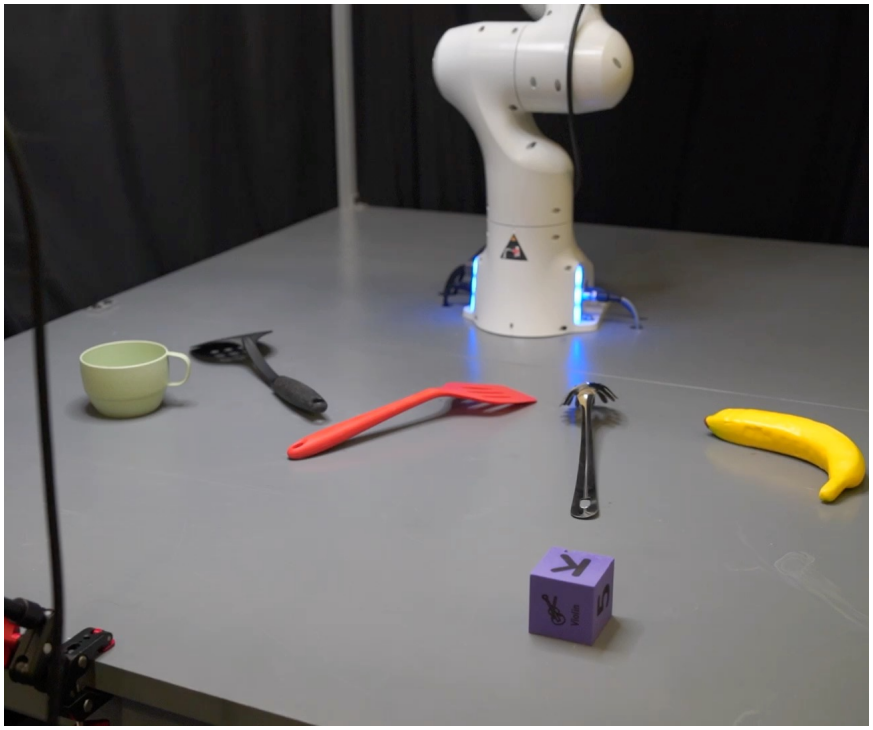
1. There is an external physical reality that is worth believing in
2. World-model + planning algorithm = policy
3. Pick representations for learning based on sample efficiency / generalization
4. Learning can make planning better
5. Planning can make learning better

What Can We Learn from One Demonstration?





What can we learn from a single demonstration

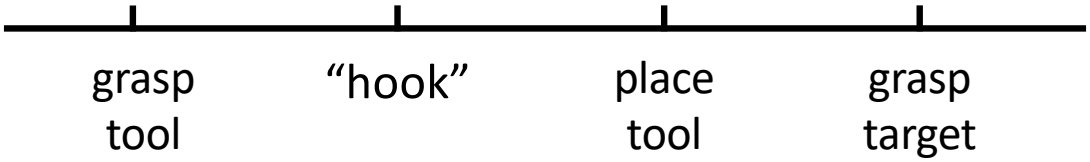


⟨ name, type, **??** ⟩

hook (r: robot, tool: object, target: object)
→ action_sequence

High-order concepts as a "program"!

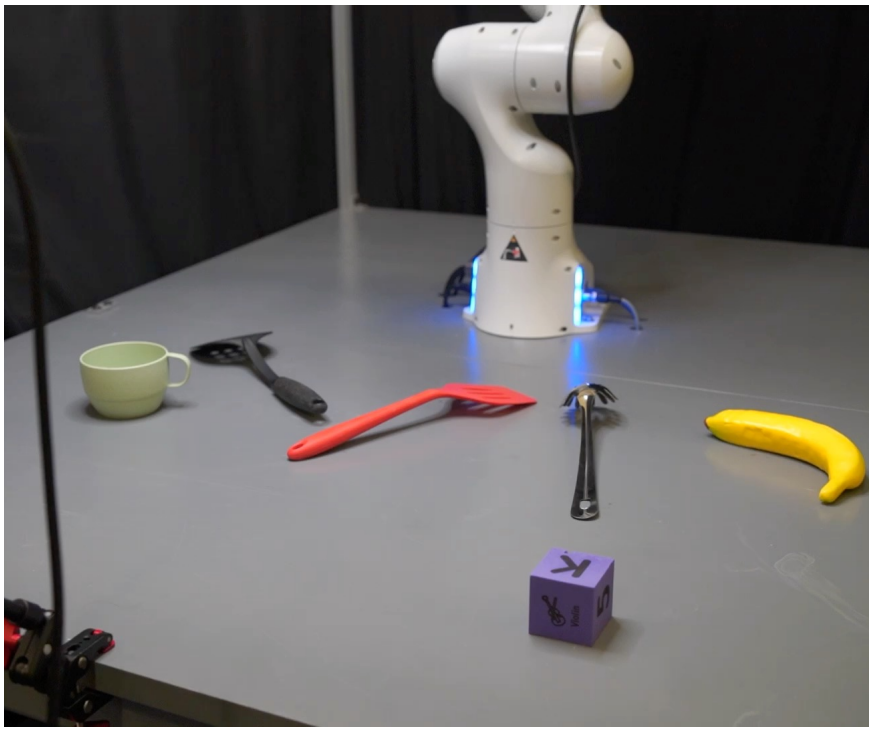
Modeled as a sequence of contacts



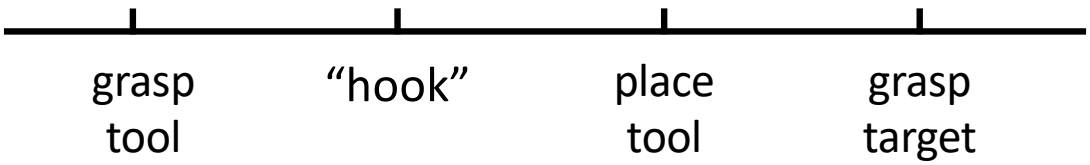
Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

Contact mode subgoals in hook-using

Key idea: model manipulation "strategies" as a program of a sequence of subgoals about contacts among objects



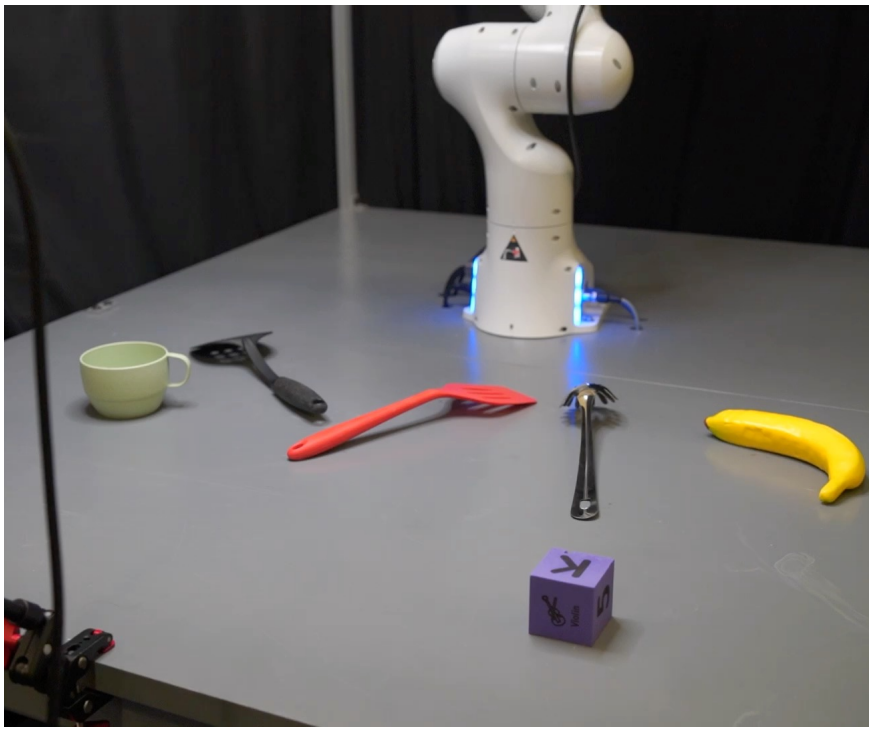
```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```



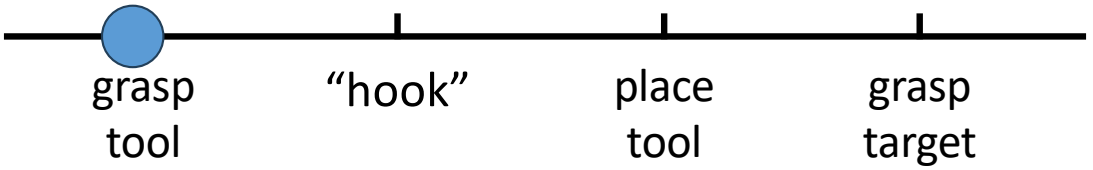
Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

Contact mode subgoals in hook-using

Key idea: model manipulation "strategies" as a program of a sequence of subgoals about contacts among objects



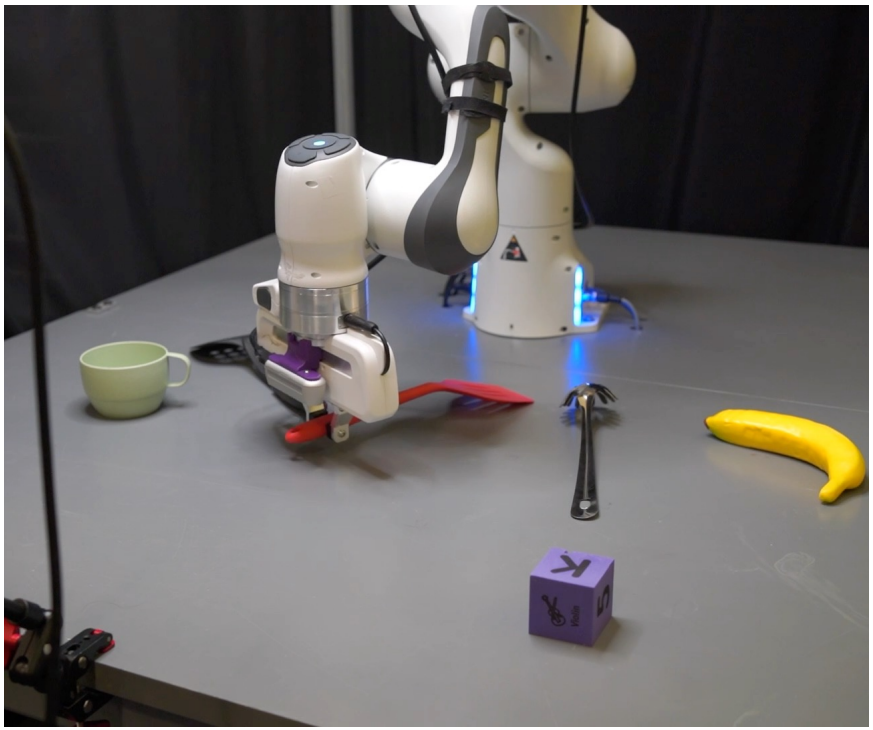
```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```



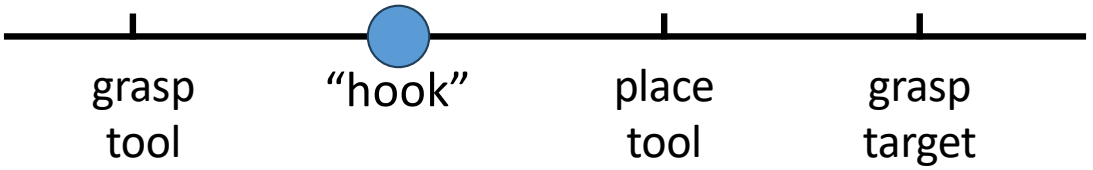
Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

Contact mode subgoals in hook-using

Key idea: model manipulation "strategies" as a program of a sequence of subgoals about contacts among objects



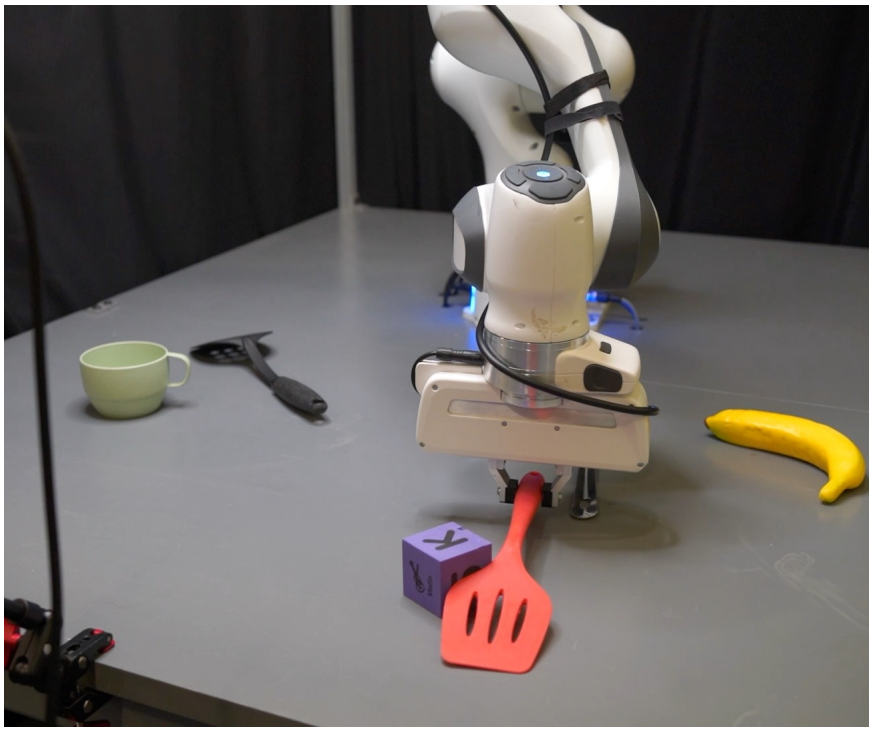
```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```



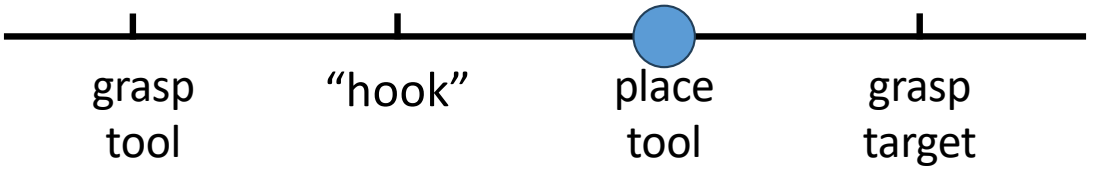
Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

Contact mode subgoals in hook-using

Key idea: model manipulation "strategies" as a program of a sequence of subgoals about contacts among objects



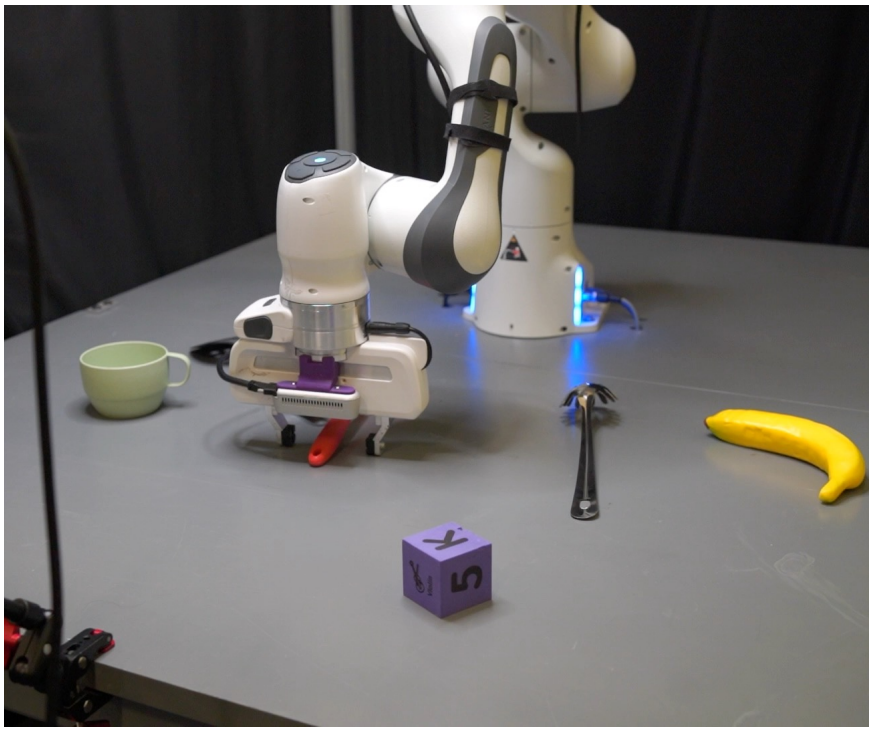
```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```



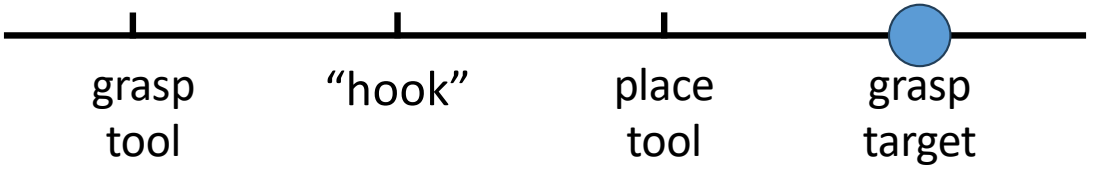
Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

Contact mode subgoals in hook-using

Key idea: model manipulation "strategies" as a program of a sequence of subgoals about contacts among objects



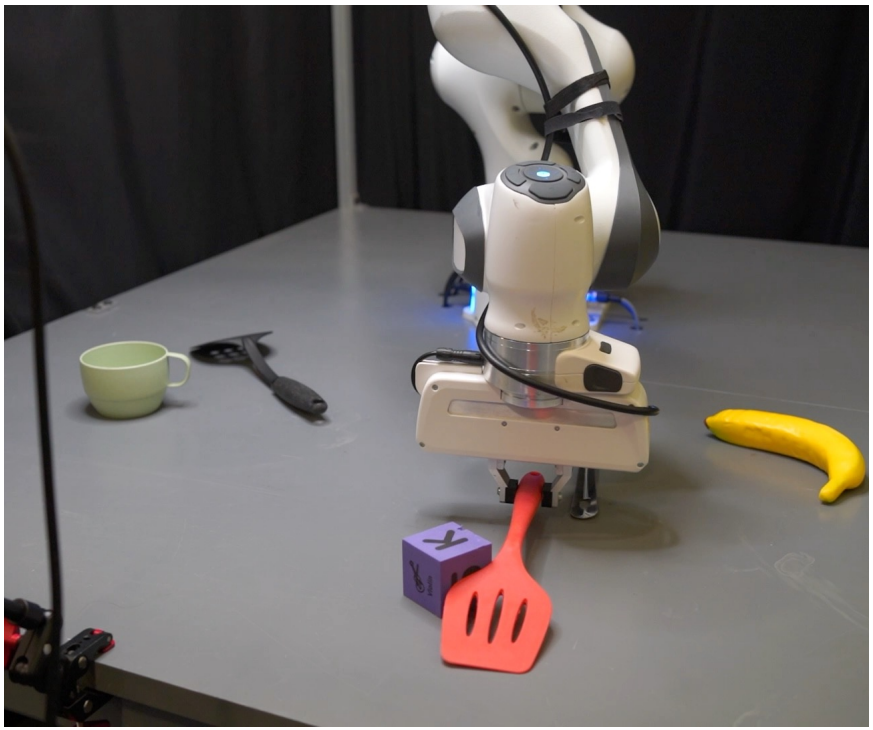
```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```



Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

Contact mode subgoals in hook-using

Key idea: model manipulation "strategies" as a program of a sequence of subgoals about contacts among objects



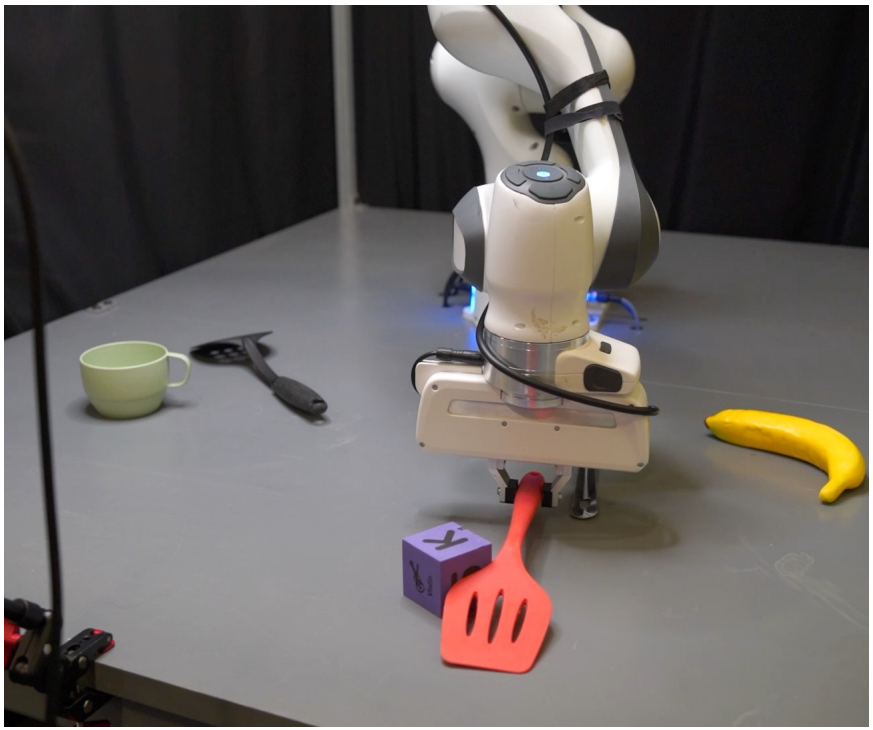
```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```

The sequence of subgoals can be learned from a single demonstration!

Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

Contact mode subgoals in hook-using

Key idea: model manipulation "strategies" as a program of a sequence of subgoals about contacts among objects



```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```

The sequence of subgoals can be learned from a single demonstration!

Concrete values for grasping and trajectories vary across instances

Mao, Lozano-Perez, Tenenbaum, Kaelbling. CoRL 2023. Learning Reusable Manipulation Strategies.

New operation can be used as a component in future plans!

Goal: in(cube, cup)



Generalization to new tools, without any training or object models

Learning Contact Constraints and Values

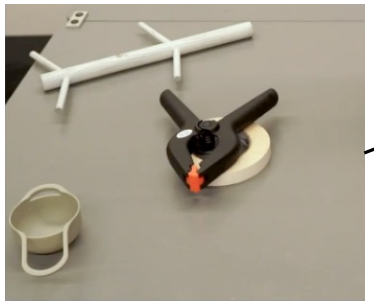


Single Demo:

Using a "hook"

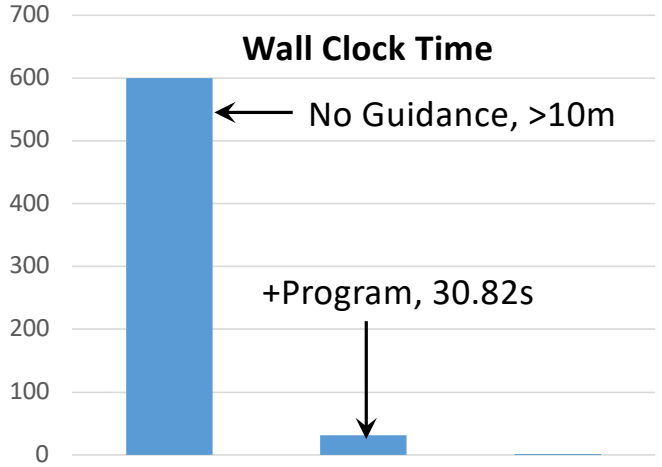
The high-level concept as a constraint set

```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```



New Instance

Sampling-Based Opt. following the Program



Yuyao Liu, Jiayuan Mao, ... One-Shot Manipulation Strategy Learning by Making Contact Analogies

Learning Contact Constraints and Values



Single Demo:

Using a "hook"

The high-level concept as a constraint set

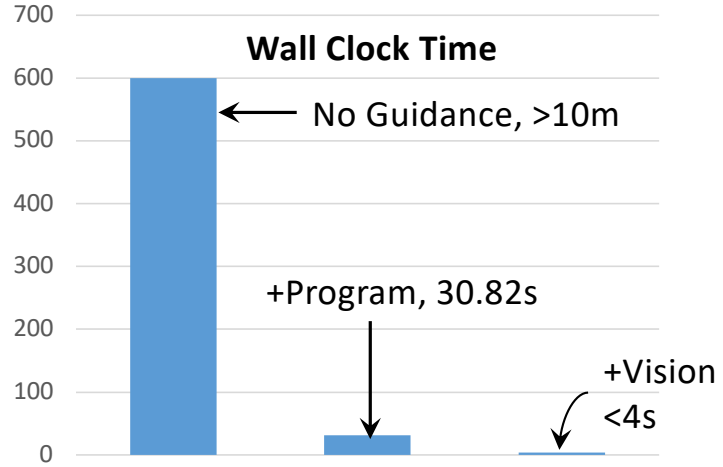
```
behavior hook(target, tool):
  body:
    achieve holding(tool, ?grasp1)
    move-with-contact(tool, target, ?traj)
    place_down(tool)
    grasp(target, ?grasp2)
```



New Instance

Sampling-Based Opt. following the Program

Leveraging Additional Visual Cues



*: The demonstration is given in simulation so we can extract contact points directly

5 key ideas

1. There is an external physical reality that is worth believing in
2. World-model + planning algorithm = policy
3. Pick representations for learning based on sample efficiency / generalization
4. Learning can make planning better
5. Planning can make learning better

Planning to practice:
Self-motivated, fully independent, task driven learning

Robot

- Has a set of parameterized skills
- Knows a task distribution
- Is not very good at deploying those skills
- Has some free time on its hands
- Decides to practice:
  - Choose to practice skills that are important for its tasks and that it's bad at
  - Plans, using its current skills:
    - to decide which skills are important for its tasks
    - to set up the environment so that it can practice the skill
  - Plan to **practice important** skills you think you **can improve at** at!
  - Plan to **use** skills you think you are **good** at!



Boston Dynamics AI INSTITUTE

Nishanth Kumar, Tom Silver, Willie McClinton, Linfeng Xiao, BDAII colleagues, RSS 2024


Cleanup Playroom Environment

Enormous questions remain!

- Deciding on interfaces:
  - Language and images are good (and have agreed-upon semantics)
  - Will they suffice? What about 3D? Or configuration space?

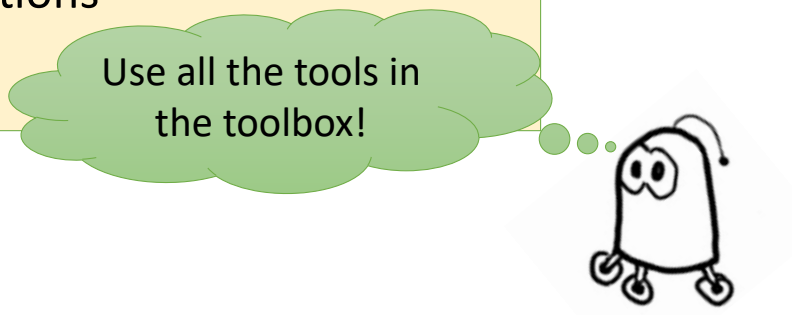
- How can we get good uncertainty quantification in all models so we can combine them dynamically and effectively online?

- How can we design flexible memory systems for space and objects that play well with other modules?

- How can we address all of these problems when there is substantial, long-term partial observability?

So, let's engineer **general-purpose** rational robots

- Start with built-in fundamental learning and inference algorithms
- Build and pre-train general, compositional world models at design time
- Continually and compositionally improve and augment these models at runtime
- Perform runtime rational inference to select actions



Huge thanks to students and colleagues!

- Adi Agarwal
- Ashay Athalye
- Rohan Chitnis
- Aidan Curtis
- Xiaolin Fang
- Caelan Garrett
- Leslie Kaelbling
- Nishanth Kumar
- Jiayuan Mao
- Willie McClinton
- Tom Silver
- Josh Tenenbaum

We gratefully acknowledge support from
- NSF grant 2214177
- AFOSR grant FA9550-22-1-0249
- ONR MURI grant N00014-22-1-2740
- ARO grant W911NF-23-1-0034
- the MIT-IBM Watson Lab
- the Boston Dynamics Artificial Intelligence Institute
- the MIT Quest for Intelligence